

基于价格指数、加性分解与智能优化的蔬菜自动定价及补货决策

摘要

生鲜蔬菜商品保鲜期短，商超须在每日凌晨进货前，于不确切知晓当日具体单品与进货价格的情况下，完成补货与定价决策。本文以某商超 2020 年 7 月 1 日至 2023 年 6 月 30 日的蔬菜销售流水、批发价格及损耗率数据为基础，构建了一套从需求规律挖掘、价格弹性估计到分层优化决策的系统性建模框架。

问题一中首先对原始数据进行清洗：以 0 填补无销售记录日期的缺失值，剔除累计销量低于 25%分位数的滞销单品，并采用**箱线图法**识别和修正异常值。在此基础上，分别绘制周销量与月销量柱状图，从时序角度直观考察销量的周期性波动；进一步运用 **Kolmogorov-Smirnov 检验**判断不同品类及单品销量是否来自同一分布，并据此进行品种相关性分类；最后选取销量最高的 15 个单品，计算其**皮尔逊相关系数**，量化高销量单品间的关联强度，为问题二中品类层面的聚合提供统计依据。

问题二为解决价格与销量相互影响的内生性问题，先将单品数据按品类合并，并以销量为权重构建**加权平均价格指数**，消除品类内部结构调整的测量偏误。随后引入 **Prophet 时间序列分解模型**，将各品类历史销量拆解为趋势项、季节性周期项、节假日脉冲项及价格驱动项，从而剥离天气、节气、消费习惯等混杂因素对需求的影响。在纯化价格效应后，采用**最小二乘回归**估计各品类价格对销量的负向边际影响系数。基于此经济参数，以商超未来一周总收益最大化为目标，构建品类层的补货与定价优化模型，由于目标函数嵌套 Prophet 结构而不可导，引入**模拟退火算法**进行迭代求解，输出各品类每日补货总量与最优定价。

问题三根据 2023 年 6 月 24 日至 30 日的批发价数据筛选出次日可采购的蔬菜单品，结合问题二已估计的销量-价格线性关系，并纳入附件 4 给出的损耗率，构建以当日总利润最大化为目标的**混合整数二次规划模型**。模型以 0-1 变量控制单品是否入选，确保上架品种总数在 27 至 33 个之间；以 2.5 千克为最小陈列量下限约束进货量。针对该模型的混合整数特性，采用**差分进化算法**进行全局寻优，最终输出 7 月 1 日各单品的具体进货量与定价策略。

问题四跳出建模本身，以逆向视角审视现有数据体系的不足。本文建议商超补充采集**四类辅助数据**：竞争对手价格、天气与节气信息、商品品质等级、促销活动标记。这些数据能够增强需求预测中外生冲击因素的刻画能力，为定价决策提供更稳健的边界约束，并有效降低因信息不充分导致的库存积压与利润损耗风险，最终形成从数据采集、需求建模、优化决策到反馈补采的完整闭环。

关键词：Prophet 时间序列分解最小二乘回归模拟退火算法差分进化算法

一、问题重述

1.1 问题背景

生鲜蔬菜商品具有保鲜期短、品相易变的特点，大部分品种如当日未售出，隔日即无法再售，因此商超通常需根据历史销售与需求情况每日进行补货。然而，由于蔬菜品种众多、产地各异，且进货交易时间通常在凌晨 3:00 至 4:00，商家须在不确切知道当日具体单品和进货价格的情况下做出补货决策。在定价方面，蔬菜一般采用成本加成定价方法，对运损和品相变差的商品进行打折销售。从需求侧看，蔬菜销售量与时间存在关联关系；从供给侧看，蔬菜供应品种在 4 月至 10 月较为丰富，商超销售空间的限制使得合理的销售组合极为重要。

附件 1 给出了某商超经销的 6 个蔬菜品类的商品信息；附件 2 和附件 3 分别给出了该商超 2020 年 7 月 1 日至 2023 年 6 月 30 日各商品的销售流水明细与批发价格数据；附件 4 给出了各商品近期的损耗率数据。需基于上述附件及实际情况，解决以下四个问题。

1.2 问题要求

问题 1 要求分析蔬菜各品类及单品销售量的分布规律及相互关系。

问题 2 要求以品类为单位制定补货计划，分析各蔬菜品类的销售总量与成本加成定价的关系，并给出各品类未来一周（2023 年 7 月 1 日至 7 日）的日补货总量和定价策略，使商超收益最大。

问题 3 要求在销售空间有限的前提下制定单品的补货计划，将可售单品总数控制在 27 至 33 个，且各单品订购量满足最小陈列量 2.5 千克的要求，根据 2023 年 6 月 24 日至 30 日的可售品种，给出 7 月 1 日的单品补货量和定价策略，在尽量满足市场对各品类需求的前提下使商超收益最大。

问题 4 要求从完善决策的角度出发，提出商超还需采集的补充数据，并说明这些数据对解决上述问题的具体帮助。

二、问题分析

2.1 问题 1 的分析

问题 1 的核心在于从数据层面揭示蔬菜销售的底层规律。蔬菜品类与单品繁多，不同品种间的销售行为可能存在替代、互补或独立等关联关系，这些关系直接影响后续补货与定价决策的准确性。因此，本问题的分析应从三个维度展开：

其一，数据清洗与质量把控。原始销售数据涵盖三年周期，必然存在无销售记录的缺失日期以及异常销售记录。需以合理方式处理缺失值（如以 0 填补）、剔除长期滞销单品，并采用箱线图法识别和修正异常值，为后续分析奠定可靠的数据基础。

其二，销量分布规律的刻画。通过绘制周销量与月销量柱状图，可从时序角度直观考察销量的周期性波动特征，识别是否存在以周为单位的规律性变化以及季节性趋势。

其三，品类与单品间相互关系的量化。不同品类或单品之间可能存在销量上的关联——或同涨同跌，或此消彼长。一方面，运用 Kolmogorov-Smirnov 检验判断不同品种销量是否来自同一分布，据此进行品种类型的相关性分类；另一方面，选取销量最高的 15 个单品计算皮尔逊相关系数，量化高销量单品间的关联强度。上述分析为问题二中品类层面的数据聚合提供了统计依据。

2.2 问题 2 的分析

问题 2 将分析视角从描述性统计提升至因果推断层面，核心任务是建立销量与定价之间的定量关系，并据此做出优化决策。分析中需重点攻克以下难点：

难点一：价格与销量间的内生性问题。销量受价格影响，但价格本身也可能受销量预期的影响，二者互为因果。若直接回归将产生估计偏误。应对策略为：先将单品数据按品类合并，以销量为权重构建加权平均价格指数，消除品类内部结构调整带来的测量偏误。

难点二：混杂因素的剥离。销量波动受多重因素驱动——长期趋势、季节性周期、节假日脉冲、天气与消费习惯等，价格只是其中之一。若不加分离直接估计价格效应，将混淆各因素的影响。为此引入 Prophet 时间序列分解模型，将各品类历史销量精准拆解为趋势项、季节性周期项、节假日脉冲项及价格驱动项，有效剥离混杂因素。

难点三：优化模型的求解。在纯化价格效应后，采用最小二乘回归估计各品类价格对销量的负向边际影响系数，以此作为需求侧的经济学参数。在此基础上，以商超未来一周总收益最大化为目标构建品类层优化模型。由于目标函数嵌套了 Prophet 分解结构而无法直接求导，需引入模拟退火算法进行迭代寻优，输出各品类每日补货总量与最优定价。

2.3 问题 3 的分析

问题 3 将决策颗粒度从品类进一步下沉至单品，面临的核心挑战在于：在有限的陈列空间约束下（可售单品 27 至 33 种），如何从 61 种可采购品种中筛选最优组合并确定各自的进货量与售价。分析要点如下：

约束条件方面，需同时满足三个层面的限制：单品选择约束（0-1 变量控制是否入选）、总数约束（27 至 33 个）和最小陈列量约束（各单品不低于 2.5 千克）。

需求刻画方面，需结合问题 2 已估计的销量-价格线性关系，并纳入附件 4 给出的损耗率，将实际可售数量与进货数量区分开来。

求解策略方面，该问题本质上是一个含整数变量的非线性优化问题——目标函数为当日总利润最大化，决策变量既包括 0-1 选择变量又包括连续的价格与进货量变量，属于混合整数二次规划模型。针对其非凸、含整数的特点，采用差分进化算法进行全局寻优，最终输出 7 月 1 日各单品的最优进货量与定价策略。

2.4 问题 4 的分析

问题 4 与前三个问题性质不同，它是一个开放性的数据需求分析问题，要求脱离数据框架，以决策者的视角审视当前信息体系的不足。

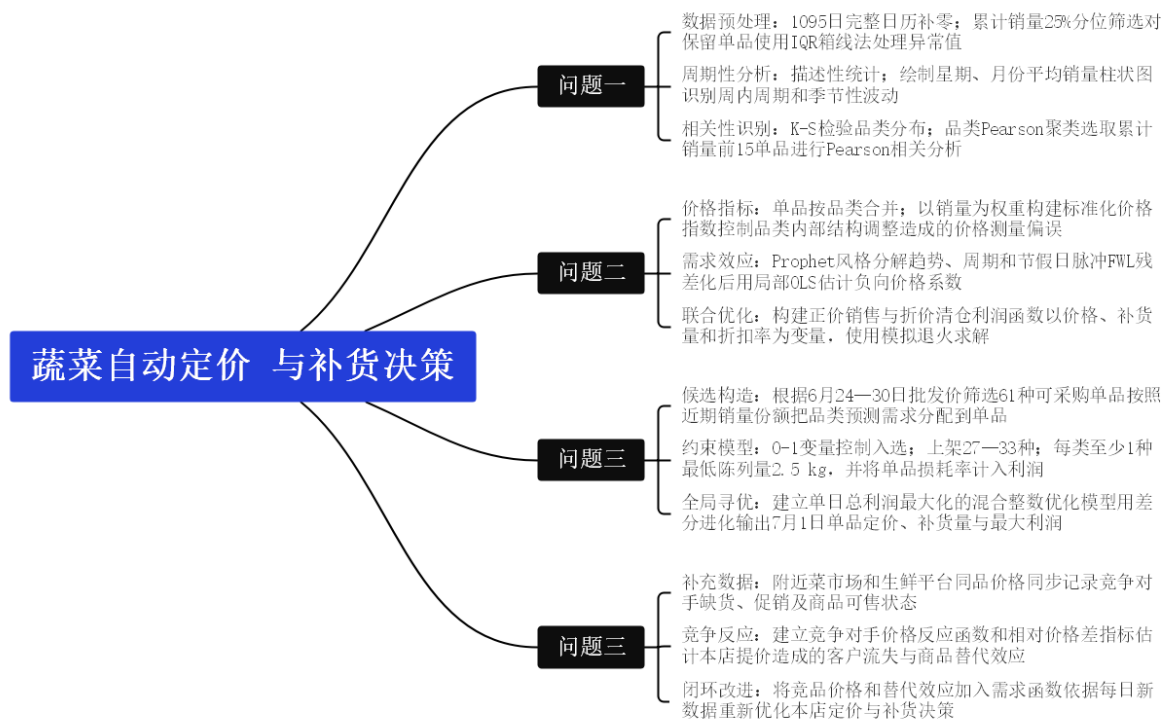


图 1 四问模型结构及算法衔接

图 1 展示了本文从规律识别到经营决策的整体建模路径。问题一首先对销售数据进行补零、筛选和异常值处理，并识别不同品类及重点单品的周期性和相关关系；问题二在品类层面构建销量加权价格指数，通过加性分解和局部线性回归估计价格对销量的边际影响，再利用模拟退火确定未来 7 日的定价与补货方案；问题三进一步将品类需求分配至单品，在品种数量、最低陈列量和损耗率等约束下，采用差分进化确定 7 月 1 日的单品组合；问题四则通过引入竞争对手价格和可售状态，形成信息采集—需求修正—重新优化的闭环。四问之间具有由统计分析、需求估计到经营优化的递进关系。

三、模型假设

1、局部线性可微性。在历史价格区间内，各品类销量对价格的偏导数存在且近似为常数，即需求量与价格满足线性关系，保证可通过最小二乘估计价格系数。

2、时间序列分量可加性。各品类销量时序可分解为趋势、季节、节假日与价格四个独立分量，且各分量相互正交，保证 Prophet 分解的数学封闭性与参数可辨识性。

3、损耗率为固定比例。各单品实际可售量等于进货量乘以 $(1-\text{损耗率})$ ，损耗率由附件 4 给定为常数，不随进货批量、季节等因素变动。

4、零库存结转。当日未售出商品当日核销，不进入次日库存，因此每日进货量与当日可售量相等，库存状态变量退化为零。

5、需求函数全局有界。优化变量（价格与进货量）在可行域内取值时，需求函数不出现退化情形（销量恒为零或销量趋于无穷），保证目标函数在紧集上存在有限最优解。

6、零交叉价格弹性。各品类及单品之间的需求相互独立，某一商品价格调整不引发其他商品需求量的变动，价格效应不通过品类间传导。

四、符号说明

符号	含义	单位
i, k, t	单品、品类和日期索引	—
y_{it}, y_{kt}	单品或品类日净销量	kg
p_{it}, P_{kt}	单品售价与品类价格指数	元/kg、指数元
c_{it}, C_{kt}	单品批发价与品类成本指数	元/kg、指数元
q_{it}, q_{kt}	单品或品类补货量	kg
r_i, r_k	单品或品类损耗率	—

Bkt	参考价格下的加性模型 基准需求	kg
β_k, b_i	品类和单品价格边际系数	kg/元
thetakt	临期折价系数	—
x_i	单品是否入选	0—1
S^R, S^D	正价销量与折扣销量	kg
p^{cit}	匹配竞品的价格	元/kg

五、数据预处理与描述性分析

5.1 数据结构与清洗

我们使用将数据类型强制转换和规范化编码方法统一各附件的数据格式。首先，将单品编码和分类编码统一转换为字符串，并删除编码前后的空格；其次，采用日期解析函数将销售日期和批发价格日期转换为标准日期类型，并将日期统一表示为年一月一日格式；再次，采用数值强制转换方法将销量、销售单价、批发价格和损耗率转换为浮点型数据。所有销量和补货量统一以千克为单位，销售单价及批发价格统一以元/千克为单位，损耗率则由百分数除以 100 转换为 0 至 1 之间的小数。此外，根据销售类型将原始交易划分为正向销售和退货，并将是否打折销售转换为 0—1 变量。最后，以单品编码作为四个附件的统一连接字段，以日期+单品编码作为销售数据与批发价格数据的联合匹配键，从而保证不同附件之间字段类型、计量单位和编码规则一致。并按照销售类型将正向销售与退货记录分离。以销售日期—单品编码为单位汇总逐笔交易数据，其中日净销量等于正向销售量与退货量之差，日均售价采用正向销售量加权计算。考虑到原始销售流水并未覆盖研究期内所有日期，本文构造 2020 年 7 月 1 日至 2023 年 6 月 30 日的完整日期—单品面板，将无销售记录的销量补为 0，但不将缺失售价和批发价格补为 0。随后计算各单品三年累计净销量，以累计销量第 25 百分位数 15.2975kg 为筛选阈值，最终保留 188 个单品，删除 63 个低频单品。对于日净销量为负的观测，将其修正为 0。进一步对每个单品的正销量序列分别采用箱线图法识别异常值，以 $Q1-1.5IQR$ 和 $Q3+1.5IQR$ 为异常边界，并通过缩尾法将异常销量修正至相应边界，共修正 1942 个单品—日期观测。批发价格缺失值则在同一单品内部按日期进行线性插值，序列两端使用最近有效价格填充。经过上述处理，获得结构完整、时间连续且适合后续分布检验、相关分析和需求预测的日级面板数据。

附件 1 含 251 个单品及 6 个品类；附件 2 含 878503 条逐笔销售流水；附件 3 含 55982 条单品日批发价；附件 4 含 251 条单品损耗率及 6 条品类平均损耗率。四表均无缺失单元格和完全重复记录。销售流水中有 878042 条销售记录和 461 条退货记录，退货数量已经为负，故不再重复改号；47366 条记录带折扣标记，占 5.39%。

销售数据覆盖 2020 年 7 月 1 日至 2023 年 6 月 30 日，共 1085 个有流水日期，完整日历为 1095 天，另有 10 天没有任何流水。按照本文第一问的统一口径，建立 1095 个日期 $\times$ 251 个登记单品的完整面板，无销售记录的日期—单品组合用 0 替代。退货按附件中的负销量直接并入当日净销量；若聚合后出现负的单品日净销量，则截断为 0，避免把退货集中登记解释为负需求。该补零规则保证不同品类和单品使用相同观察窗口，但可能把闭店或缺货混入零销量，因此不将其解释为真实零需求。

5.2 低销量筛选与异常值处理

为剔除长期销售频率较低、零销量比例过高的单品，本文采用基于第 25 百分位数的低销量筛选方法。首先计算每个单品在三年研究期内的累计净销量，再以全部单品累计销量的第 25 百分位数作为筛选阈值，计算公式如下：

$$Q_i = \sum_{t=1}^{1095} y_{it}, \quad \tau = P_{25}\{Q_i\}.$$

计算得到：

$$\tau = 15.2975 \text{ kg. } \tau = 15.2975 \text{ kg.}$$

为识别单品日销量中的极端异常记录，本文采用 Tukey 箱线图算法。考虑到各单品销量规模存在明显差异，本文分别对每个单品的正销量序列计算异常值边界，而不是对全部单品使用统一阈值。计算公式如下：

$$IQR_i = Q_{3i} - Q_{1i}, \quad L_i = \max\{0, Q_{1i} - 1.5IQR_i\}, \quad U_i = Q_{3i} + 1.5IQR_i.$$

对于箱线图算法识别出的异常值，本文采用 Winsor 缩尾算法进行修正，而不是直接删除异常日期。该方法将异常销量替换为相应的箱线图边界，计算公式如下：

$$\tilde{y}_{it} = \min\{U_i, \max\{L_i, y_{it}\}\}, \quad y_{it} > 0.$$

问题二、三在上述净销量口径上另行构造价格和成本变量：售价采用归一化销量加权价格指数，品类成本采用同口径批发价指数，折扣参数采用折扣销量占正销量的比例；

损耗率由百分数除以 100 后进入模型。描述商超销售价格相对于批发成本的加成程度，本文采用成本加成率指标衡量定价水平，计算公式如下：

$$m_{kt} = \frac{p_{kt} - c_{kt}}{c_{kt}}.$$

计算结果表明，251 个登记单品三年累计净销量的第 25 百分位数为 15.2975 kg。以此为阈值，共删除 63 个累计销量低于阈值的低频单品，保留 188 个单品进入问题一分析。保留单品占全部登记单品的 74.90%。筛选后，三年累计销量最高的单品依次为**芜湖青椒（1）**、**净藕（1）**和**西兰花**，累计销量分别为 26802.46 kg、26794.50 kg 和 26719.91 kg，三者明显属于商超的**核心畅销单品**。

从畅销单品的品类构成看，前 15 名单品中花叶类和辣椒类出现次数最多，说明这两类商品不仅品类总销量较大，而且具有较多销量稳定的核心单品。芜湖青椒（1）、净藕（1）和西兰花虽然分属三个不同品类，但累计销量非常接近，均超过 2.67 万 kg，应作为商超日常重点保障供应的商品。

5.3 品类销量初步特征

六类日销量均表现出不同程度的右偏和波动。**花叶类规模最大**，清洗后日均销量 172.94kg；辣椒类与食用菌分别为 80.14kg 和 64.95kg；花菜类、水生根茎类和茄类分别为 37.11kg、35.94kg 和 19.45kg。水生根茎类变异系数 0.753、偏度 1.158，说明相对波动和右尾都较明显；茄类偏度 0.560，相对更接近对称。

分类名称	均值	标准差	最小值	25%分位数	中位数	75%分位数	最大值	变异系数	偏度
花叶类	172.941	71.966060		125.8695	169.7514	211.0939	623.0774	40.4161	310.8541
花菜类	37.1102	19.982990		23.095	33.858	48.4825	99.0488	80.5384	770.7163
水生根茎类	35.9392	27.0753	20	14.9255	30.005	50.4765	142.3965	50.7533	631.1581
茄类	19.4475	611.1605	10	11.5385	18.14	26.109	57.2973	380.5738	770.5595
辣椒类	80.1421	642.3058	90	50.48	72.08	99.3559	4298.4551	10.5278	861.2218
食用菌	64.9532	636.2807	80	38.73	56.776	84.2386	9227.9205	10.5585	681.0531

六、问题一：销量分布、周期性与相关关系

6.1 周期性销量规律

为分析各蔬菜品类销量的星期周期和月份周期，本文采用分组平均统计方法，分别按照星期和月份对品类日销量进行分组，计算公式如下：

$$\overline{y}_{k,w} = \frac{1}{n_w} \sum_{t:W_t=w} y_{kt}, \quad \overline{y}_{k,m} = \frac{1}{n_m} \sum_{t:M_t=m} y_{kt}.$$

式中， y_{kt} 表示品类 k 在日期 t 的日销量； W_t 和 M_t 分别表示日期 t 所属的星期和月份； n_w 和 n_m 分别表示相应分组包含的日期数； $\overline{y}_{k,w}$ 和 $\overline{y}_{k,m}$ 分别表示星期平均销量和月份平均销量，单位为 kg/日。通过比较不同星期和月份的平均日销量，可以判断各品类是否存在周末效应和季节性波动。

星期柱状图显示明显的周末效应：花叶类、水生根茎类、辣椒类和食用菌均在星期六达到一周最高，分别为 210.97、47.98、101.69 和 83.90kg/日；花菜类与茄类在星期日最高，分别为 46.63 和 24.38kg/日。多数品类在星期二至星期四处于低位，例如花叶类星期四仅 153.13kg/日，表明周末客流或家庭采购可能带来共同提升。

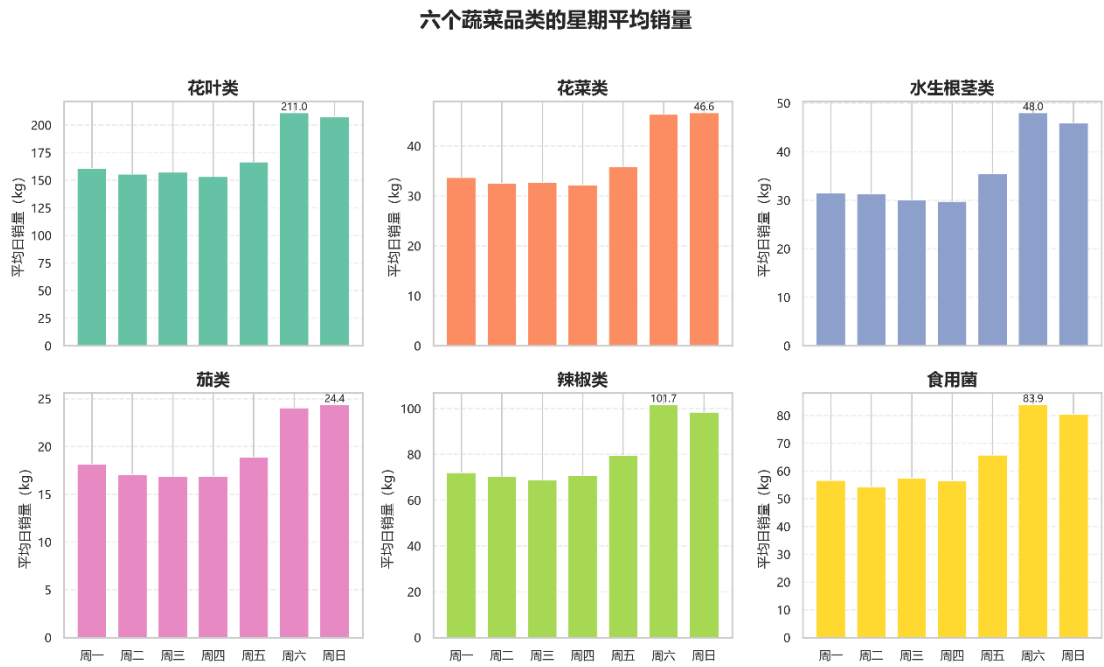


图 2 六品类星期平均销量柱状图

图 2 反映了六个蔬菜品类在一周内的平均销量变化。花叶类、水生根茎类、辣椒类和食用菌均在星期六达到周内最高销量，分别约为 210.97、47.98、101.69 和 83.90kg；花菜类和茄类则在星期日达到最高值，分别约为 46.63 和 24.38kg。多数品类在星期二至星期四销量相对较低，说明蔬菜销售具有较明显的周末效应，可能与周末客流增加和家庭集中采购有关。因此，后续销量预测和补货决策应将星期因素作为重要的季节性变量，并在周末适当提高补货量。

月份柱状图进一步显示季节差异。花叶类、花菜类和辣椒类均在 8 月达到高位，日均分别为 248.32、50.02 和 103.91kg；水生根茎类与食用菌在 1 月最高，分别为 60.77 和 97.07kg；茄类在 7 月最高，为 28.10kg。不同品类峰谷月份并不一致，说明后续需求预测需要显式考虑月份变量，不能只使用统一的年度平均量。

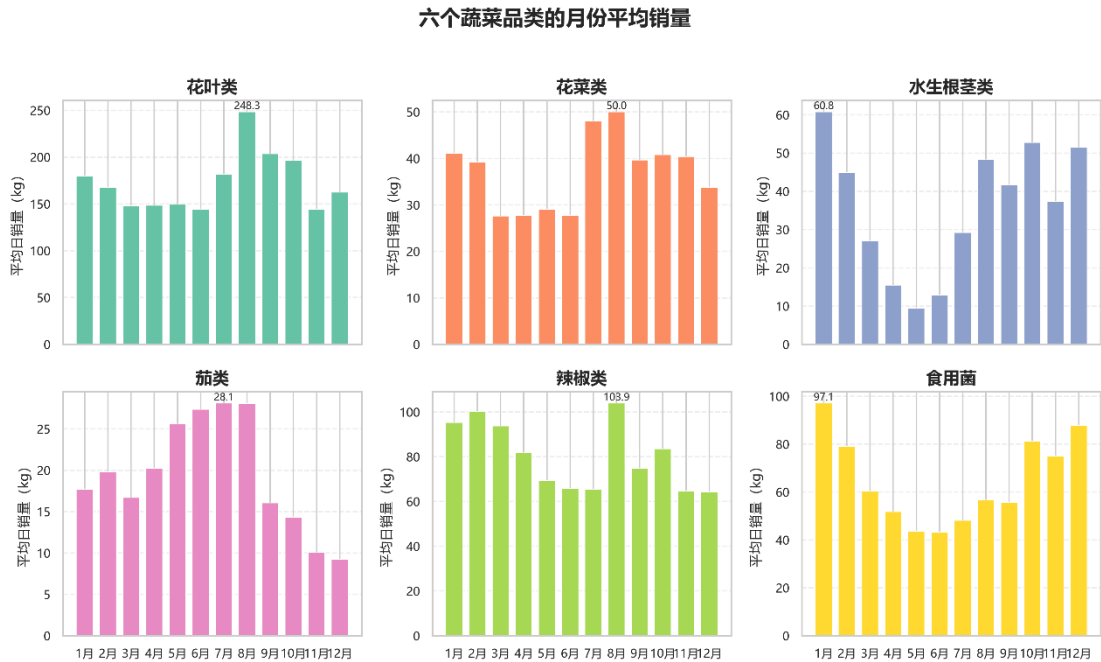


图 3 六品类月份平均销量柱状图

图 3 展示了六个品类的月度季节变化。花叶类、花菜类和辣椒类均在 8 月份处于较高销量水平，日均销量分别约为 248.32、50.02 和 103.91kg；水生根茎类和食用菌在 1 月份达到最高值，分别约为 60.77 和 97.07kg；茄类则在 7 月份达到峰值，约为 28.10kg。不同品类的峰值月份并不一致，表明各品类受到上市季节、气候条件和消费习惯的影响不同。因此，不能采用统一的年度平均销量描述全部品类，应分别加入月份或年周期变量进行预测。

6.2 品类销量分布的 K-S 检验

判断任意两个蔬菜品类的日销量是否来自同一概率分布，本文采用两独立样本 Kolmogorov-Smirnov 检验。该方法不要求销量服从正态分布，适合比较蔬菜销量这类

具有偏态和大量零值的数据。建立假设如下：

$$D_{ab} = \sup_x |\hat{F}_a(x) - \hat{F}_b(x)|.$$

在显著性水平 $\alpha=0.05$ 下，15 组两两检验的 p 值均小于 0.05，因此全部拒绝同分布原假设。花菜类—水生根茎类的 D 值最小，为 0.1662，辣椒类—食用菌次之，为 0.1854，说明这两对分布形态相对接近；花叶类—茄类 $D=0.9726$ ，差异最大。K-S 检验结论意味着六品类应分别建模或至少使用分层模型，不宜假设共同分布参数。

品类 A	品类 B	K-S 统计量	p 值	5%水平结论
花叶类	花菜类	0.894977	0	拒绝同分布
花叶类	水生根茎类	0.853881	0	拒绝同分布
花叶类	茄类	0.972603	0	拒绝同分布
花叶类	辣椒类	0.63105	#####	拒绝同分布
花叶类	食用菌	0.713242	#####	拒绝同分布
花菜类	水生根茎类	0.16621	1.28E-13	拒绝同分布
花菜类	茄类	0.427397	4.14E-90	拒绝同分布
花菜类	辣椒类	0.52968	#####	拒绝同分布
花菜类	食用菌	0.359817	2.29E-63	拒绝同分布
水生根茎类	茄类	0.338813	4.38E-56	拒绝同分布
水生根茎类	辣椒类	0.516895	#####	拒绝同分布
水生根茎类	食用菌	0.374429	1.02E-68	拒绝同分布
茄类	辣椒类	0.828311	0	拒绝同分布
茄类	食用菌	0.708676	#####	拒绝同分布
辣椒类	食用菌	0.185388	7.41E-17	拒绝同分布

6.3 品类 Pearson 相关性分类

为了衡量不同蔬菜品类日销量之间的线性同步关系，本文采用 Pearson 相关系数法，计算公式如下：

$$r_{ab} = \frac{\sum_t (y_{at} - \bar{y}_a)(y_{bt} - \bar{y}_b)}{\sqrt{\sum_t (y_{at} - \bar{y}_a)^2 \sum_t (y_{bt} - \bar{y}_b)^2}}.$$

再定义距离 $d_{ab}=1-r_{ab}$ 并采用平均连接法进行系统聚类。花叶类—辣椒类相关最高，为 0.697；花叶类—花菜类、辣椒类—食用菌、水生根茎类—食用菌的相关系数分别为

0.644、0.639 和 0.625。以聚类距离 0.55 截断，可将花叶类、花菜类、水生根茎类、辣椒类和食用菌归为第一类共同波动组，茄类单独归为第二类。茄类与水生根茎类、食用菌的相关分别为-0.091 和-0.035，说明其销量节奏与其他品类差异较大。

分类名称	花叶类	花菜类	水生根茎类	茄类	辣椒类	食用菌
花叶类	1	0.643836	0.54751	0.217698	0.697029	0.606939
花菜类	0.643836	1	0.491694	0.225673	0.488149	0.493386
水生根茎类	0.54751	0.491694	1	-0.09085	0.489832	0.624571
茄类	0.217698	0.225673	-0.09085	1	0.096439	-0.03538
辣椒类	0.697029	0.488149	0.489832	0.096439	1	0.638927
食用菌	0.606939	0.493386	0.624571	-0.03538	0.638927	1

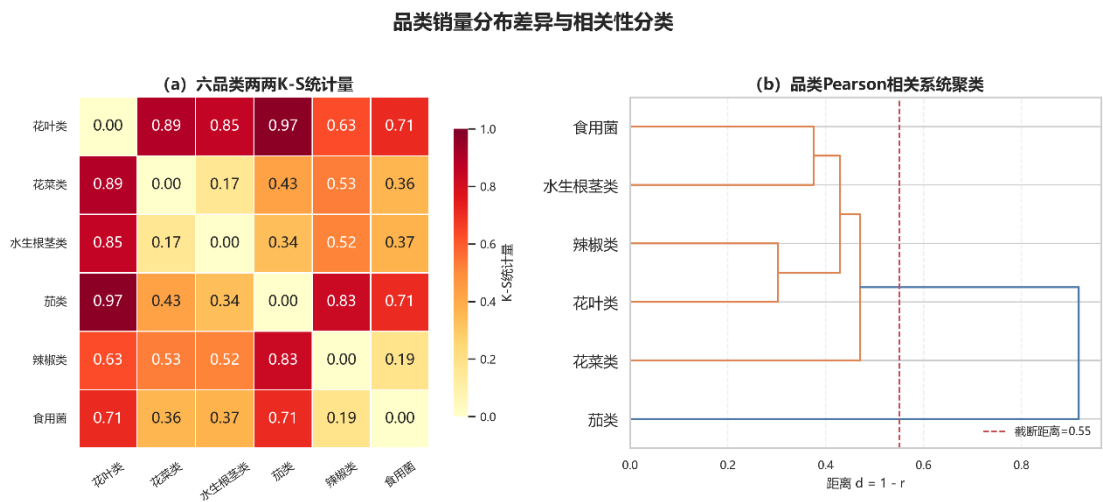


图 4 品类 K-S 统计量与 Pearson 相关系统聚类

图 4 左侧的 K-S 统计量热力图表明，六个品类的 15 组两两检验均在 5%显著性水平下拒绝销量服从同一分布的原假设。其中，花菜类与水生根茎类的 K-S 统计量最小，为 0.1662，说明二者的销量分布相对接近；花叶类与茄类的统计量最大，为 0.9726，说明二者在销量规模和分布形态上差异最明显。这一结果说明六个品类不宜采用完全相同的分布参数，应分别或分层建模。

K-S 检验和 Pearson 相关回答的是不同问题：前者检验两个品类的边际分布是否相同，后者刻画逐日同步变化。

6.4 销量前 15 单品的 Pearson 相关分析

按箱线法清洗后的三年累计销量排序，选取前 15 个单品。芜湖青椒(1)、净藕(1)和西兰花位列前三，累计销量分别为 26802.46、26794.50 和 26719.91kg。为避免相关热力图标签拥挤，以 I1—I15 表示单品，映射如下。

图中编号	单品名称	分类名称	三年累计清洗后销量
I1	芜湖青椒(1)	辣椒类	26802.46
I2	净藕(1)	水生根茎类	26794.5
I3	西兰花	花菜类	26719.91
I4	大白菜	花叶类	17423.21
I5	云南生菜	花叶类	15753.48
I6	金针菇(盒)	食用菌	14474.5
I7	云南生菜(份)	花叶类	14081.5
I8	紫茄子(2)	茄类	13070.73
I9	西峡香菇(1)	食用菌	11127.2
I10	小米椒(份)	辣椒类	10621
I11	云南油麦菜	花叶类	9922.288
I12	泡泡椒(精品)	辣椒类	9530.413
I13	云南油麦菜(份)	花叶类	8574
I14	螺丝椒(份)	辣椒类	8215
I15	青梗散花	花菜类	8116.564

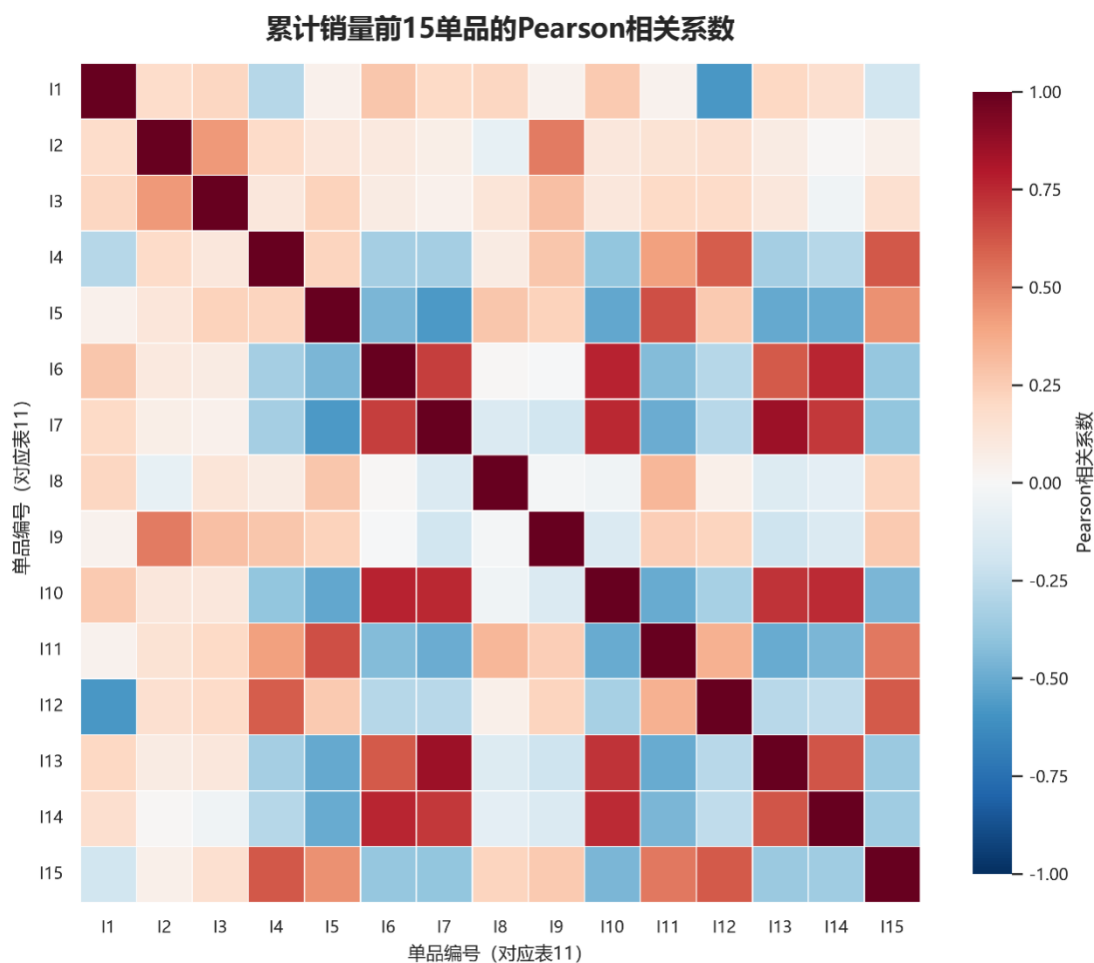


图 5 销量前 15 单品 Pearson 相关系数热力图

图 5 展示了三年累计销量最高的 15 个单品之间的 Pearson 相关系数。105 组单品对中，65 组表现为正相关，40 组表现为负相关，其中 95 组在 5%水平下显著。云南生菜（份）与云南油麦菜（份）的正相关最强，相关系数为 0.8574；芜湖青椒（1）与泡泡椒（精品）的负相关最强，相关系数为-0.5819。

105 组单品对中 65 组为正相关、40 组为负相关，95 组在 5%水平显著。最强正相关为云南生菜(份)—云南油麦菜(份)， $r=0.8574$ ；金针菇(盒)—小米椒(份)、金针菇(盒)—螺丝椒(份)分别为 0.7728 和 0.7593。最强负相关为芜湖青椒(1)—泡泡椒(精品)， $r=-0.5819$ ；云南生菜—云南生菜(份)为-0.5753。袋装/份装单品间的强正相关可能来自同期上架和共同促销，散装与份装同名商品的负相关也可能来自上架时期切换，不能仅凭相关系数认定互补或替代。

单品 A	单品 B	r	p 值	方向
云南生菜(份)	云南油麦菜(份)	0.8574	<0.001	正相关
金针菇(盒)	小米椒(份)	0.7728	<0.001	正相关

金针菇(盒)	螺丝椒(份)	0.7593	<0.001	正相关
云南生菜(份)	小米椒(份)	0.7506	<0.001	正相关
小米椒(份)	螺丝椒(份)	0.7456	<0.001	正相关
芜湖青椒(1)	泡泡椒(精品)	-0.5819	<0.001	负相关
云南生菜	云南生菜(份)	-0.5753	<0.001	负相关
云南生菜	小米椒(份)	-0.5232	<0.001	负相关
云南生菜	云南油麦菜(份)	-0.5116	<0.001	负相关
云南生菜	螺丝椒(份)	-0.5024	<0.001	负相关

6.5 本问结论

第一问形成了日期补零—低销量筛选—箱线缩尾—描述统计—周期图—K-S 分布检验—品类聚类—重点单品 Pearson 相关的完整链条。结果表明六个品类具有显著的周、月周期，且 15 组品类对均不服从同一分布；品类相关性可概括为五类共同波动、茄类相对独立，销量前 15 单品则同时存在较强正、负相关。该结论为问题二按品类分别预测、问题三考虑单品组合关系提供依据，但相关性不等价于因果关系。

七、问题二：价格效应识别与品类联合决策

7.1 归一化销量加权价格指数

为消除不同交易销量大小造成的影响，本文采用销量加权平均算法计算单品基准售价，进一步采用累计销量作为权重，计算品类基准价格，计算公式如下：

$$\bar{p}_i = \frac{\sum_t q_{it} p_{it}}{\sum_t q_{it}}, \quad \bar{p}_k = \frac{\sum_{i \in k} Q_i \bar{p}_i}{\sum_{i \in k} Q_i}.$$

再构造日期 t 的品类价格指数

$$P_{kt} = \bar{p}_k \frac{\sum_{i \in k} q_{it} (p_{it} / \bar{p}_i)}{\sum_{i \in k} q_{it}}.$$

该指数先将每个单品价格转换为相对自身基准价，再按当日销量加权，减少天然高价单品占比上升带来的测量偏误。品类批发成本指数按相同方法构造，使售价与成本处在可比较的固定品质篮子口径。无交易日的价格指数采用相邻日线性插值，只用于时间模型，不把销量由 0 改为正值。

7.2 Prophet 式加性分解

Prophet 式加性分解算法，分离长期趋势、周周期、年周期、节假日和价格变化对销量的不同影响，本文采用 Prophet 式加性分解算法，计算公式如下：

$$y_{kt} = T_k(t) + S_k^W(t) + S_k^Y(t) + H_k(t) + \beta_k(P_{kt} - P_k^0) + \varepsilon_{kt}.$$

对累计销量最高的 15 个单品计算 Pearson 相关系数，共形成 105 组单品组合。其中 65 组为正相关，40 组为负相关，95 组在 5%显著性水平下显著。

最强正相关出现在云南生菜（份）与云南油麦菜（份）之间，相关系数为 0.8574，说明二者销量具有高度同步性；金针菇（盒）与小米椒（份）、金针菇（盒）与螺丝椒（份）的相关系数分别为 0.7728 和 0.7593，也表现出较强的同向变化关系。

最强负相关出现在芜湖青椒（1）与泡泡椒（精品）之间，相关系数为-0.5819；云南生菜与云南生菜（份）的相关系数为-0.5753。散装云南生菜与份装云南生菜的负相关可能与包装形式切换、上架时期差异或消费者选择转移有关。

强正相关单品可以在客流上升时同步增加补货，但不能仅凭正相关认定其存在互补关系；强负相关单品也可能存在替代关系或上架时期错位，仍需结合库存、促销和交易购物篮数据进一步判断。

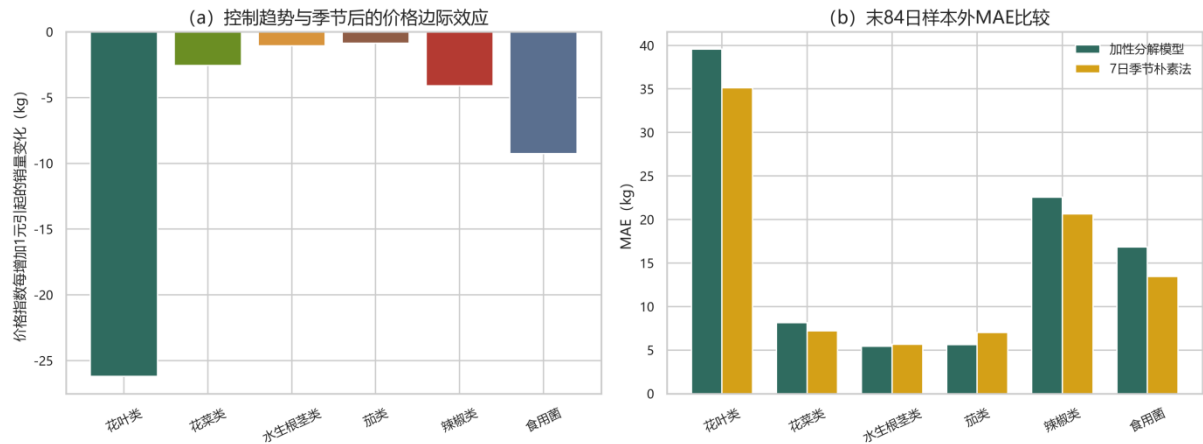
为缓解售价和销量同时受到旺季、周末及节假日影响的问题，先分别用时间设计矩阵拟合销量和价格指数，得到残差 $\hat{y}^{\perp}_{kt}$ 与 $\hat{P}^{\perp}_{kt}$ ；再在历史价格 P_{10} 至 P_{90} 局部区间内依据 Frisch-Waugh-Lovell 定理进行 OLS：

$$\hat{\beta}_k = \frac{\sum_t \hat{P}_{kt}^{\perp} \hat{y}_{kt}^{\perp}}{\sum_t (\hat{P}_{kt}^{\perp})^2}, \quad P_{kt} \in [P_{k,10}, P_{k,90}].$$

局部估计结果如下。六个无约束估计均为负，因此预设的非正经济符号约束没有实际绑定。

单品 A	单品 B	r	p 值	方向	单品 A	单品 B
云南生菜 (份)	云南油麦 菜(份)	0.8574	<0.001	正相关	云南生菜 (份)	云南油麦 菜(份)
金针菇 (盒)	小米椒 (份)	0.7728	<0.001	正相关	金针菇 (盒)	小米椒 (份)
金针菇 (盒)	螺丝椒 (份)	0.7593	<0.001	正相关	金针菇 (盒)	螺丝椒 (份)
云南生菜 (份)	小米椒 (份)	0.7506	<0.001	正相关	云南生菜 (份)	小米椒 (份)
小米椒 (份)	螺丝椒 (份)	0.7456	<0.001	正相关	小米椒 (份)	螺丝椒 (份)
芜湖青椒 (1)	泡泡椒 (精品)	-0.5819	<0.001	负相关	芜湖青椒 (1)	泡泡椒(精 品)

花叶类的条件边际影响最大，价格指数每上升 1 元，日销量平均减少约 26.20kg；食用菌和辣椒类分别减少 9.26kg 和 4.10kg。局部 R^2 仅为-0.001 至 0.078，表明控制时间因素后，价格只能解释较小部分短期销量波动，水生根茎类尤其不稳定。因此这些系数只作为历史局部区间内的经营参数，不解释为严格因果弹性，也不用于价格区间外推。



7.3 预测检验与适用边界

将最后 84 天作为时间留出集，比较加性模型和 7 日季节朴素法。加性模型在水生根茎类和茄类的 MAE 较低，其余四类的朴素法更优；说明加性分解的主要价值是分离可解释的时间与价格分量，而非保证所有品类都取得最低单一预测误差。

采用最后 84 天数据进行时间留出检验后，加性模型仅在水生根茎类和茄类上优于 7 日季节朴素法。水生根茎类的模型 MAE 为 5.43 kg，低于朴素法的 5.65 kg；茄类模型 MAE 为 5.64 kg，低于朴素法的 7.04 kg。

花叶类、花菜类、辣椒类和食用菌的加性模型 MAE 分别为 39.59、8.14、22.56 和 16.85 kg，均高于相应朴素法。因此，加性模型的主要作用是分解趋势、季节和价格影响，而不是保证所有品类均获得最低预测误差。实际应用中应根据滚动检验结果，在加性模型和季节朴素法之间动态选择。

单品 A	单品 B	r	p 值	方向
云南生菜(份)	云南油麦菜(份)	0.8574	<0.001	正相关
金针菇(盒)	小米椒(份)	0.7728	<0.001	正相关
金针菇(盒)	螺丝椒(份)	0.7593	<0.001	正相关
云南生菜(份)	小米椒(份)	0.7506	<0.001	正相关
小米椒(份)	螺丝椒(份)	0.7456	<0.001	正相关
芜湖青椒(1)	泡泡椒(精品)	-0.5819	<0.001	负相关

天气、节气和临时促销等变量未在附件中完整提供，无法声称已经被剔除；它们与其他未观测冲击共同留在残差中。若以后获得所在地和天气站数据，可把温度、降雨和湿度作为额外回归量加入同一加性结构。

7.4 正价一折扣两阶段利润模型

在历史价格的局部区间内，本文采用非负截断线性需求模型描述价格对正价需求的影响，计算公式如下：

$$D_{kt}^R(p) = \max\{0, B_{kt} + \beta_k(p - P_k^0)\}.$$

本文采用需求与供给取小原则计算正价实际销量，并计算正价销售后的剩余数量，公式如下：

$$S_{kt}^R = \min\{D_{kt}^R, A_{kt}\}, \quad L_{kt} = \max\{A_{kt} - S_{kt}^R, 0\},$$

$$D_{kt}^D = \max\{0, h_k B_{kt} + |\beta_k|(1 - \theta_{kt})p_{kt}\}, \quad S_{kt}^D = \min\{L_{kt}, D_{kt}^D\}.$$

h_k 为历史折扣销量占比， θ_{kt} 属于 $[0.60, 0.95]$ 。以近 7 日归一化批发成本指数均值 C_{kt} 作为预测成本，单日利润为

$$\Pi_{kt} = p_{kt}S_{kt}^R + \theta_{kt}p_{kt}S_{kt}^D - C_{kt}q_{kt}.$$

模拟退火算法得到未来 7 天总补货量 3700.42 kg，模型期望总利润 12312.62 元，平均折价系数为 0.830。花叶类的补货量最大，占七日总补货量的 44.47%，说明其仍是商超最主要的销量型品类；辣椒类预计利润最高，为 4496.02 元，占七日总利润的 36.52%，说明辣椒类是利润贡献最高的品类。

花叶类虽然补货量最大，但预计利润低于辣椒类，说明畅销品类不一定是利润最高的品类。茄类补货量只有 164.44 kg，却产生 1100.80 元预计利润，表明其单位销量利润相对较高。水生根茎类预计利润最低，为 338.11 元。

7.5 未来 7 日决策结果

品类	7 日平均价格指数	7 日补货量/kg	平均折价系数	预计利润/元
水生根茎类	11.78	200.50	0.950	338.11
花叶类	6.65	1645.57	0.855	3227.76
花菜类	12.80	326.85	0.890	962.07
茄类	12.48	164.44	0.722	1100.80

辣椒类	12.33	778.06	0.787	4496.02
食用菌	9.51	585.00	0.776	2187.86

日期	总补货量/kg	正价销量/kg	折扣销量/kg	预计利润/元
2023-07-01	646.62	475.44	96.46	2179.69
2023-07-02	629.99	460.61	96.57	2122.53
2023-07-03	469.42	324.12	91.10	1542.96
2023-07-04	462.72	318.32	90.94	1518.61
2023-07-05	475.20	329.36	90.97	1566.71
2023-07-06	476.28	328.63	92.68	1569.66
2023-07-07	540.19	384.11	93.77	1812.46

七日总补货量为 3700.42kg，模型期望总利润 12312.62 元，平均折价系数 0.830。最优价格多数达到历史 P90 附近，说明在当前局部线性需求与零残值假设下模型偏好较高正价、随后对剩余量打折。该边界现象也是风险提示：实际实施时应设置人工审核和小步调价，不应把历史条件关联直接解释为可无限复制的提价收益。

八、问题三：单品选择、定价与补货

8.161 种采购候选与单品需求

附件 3 在 2023 年 6 月 24 日至 30 日共有 61 个不同单品编码出现批发价记录，本文将其作为 7 月 1 日可采购单品的代理。批发成本取这 7 日均价；单品参考售价优先取近 28 日销量加权售价，无记录时使用全期加权售价，仍缺失时采用批发价的 1.35 倍。

设 w_i 为候选单品在所属品类近 28 日销量份额；对近期无销量的候选，使用全期份额的 5% 并加 0.01 平滑，随后在品类内归一化。问题二在 7 月 1 日参考价格下的品类需求 B_k 分配为 $a_i = w_i B_k$ ，单品价格斜率设为 $b_i = w_i \beta_k$ ，因此同一品类所有候选同时等额调价时，单品斜率之和仍等于品类斜率：

$$D_i(p_i) = \max\{0, a_i + b_i(p_i - p_i^0)\}, \quad \sum_{i \in k} b_i = \beta_k.$$

8.2 混合整数二次规划

令 x_i 表示单品是否入选， p_i 、 q_i 和 s_i 分别表示售价、补货量和预计销量。模型为

$$\max \Pi = \sum_i (p_i s_i - c_i q_i),$$

$$27 \leq \sum_i x_i \leq 33,$$

$$2.5x_i \leq q_i \leq M_i x_i, \quad 0 \leq s_i \leq (1 - r_i)q_i,$$

$$0 \leq s_i \leq a_i + b_i(p_i - p_i^0), \quad p_i^L x_i \leq p_i \leq p_i^U x_i.$$

目标中的 $p_i s_i$ 为二次项， x_i 为整数变量，故该模型属于非凸混合整数二次规划。由于未售商品无残值且采购成本为正，在给定选品与价格后，超出需求的补货不会增加利润，故最优补货可恢复为

$$q_i^* = \max\{2.5, D_i(p_i)/(1 - r_i)\}x_i.$$

优化结果得到 33 种入选单品的总补货量为 429.06 kg，损耗调整后的预计销售量为 392.13 kg，模型期望利润为 1414.51 元。**云南生菜（份）的补货量最高**，是 7 月 1 日需求量最大的入选单品；但**西兰花的预计利润最高**，为 290.76 元，明显高于其他单品。因此，销量最高的单品与利润贡献最高的单品并不完全相同。单品选择不能只依据历史销量，还需要**综合售价、批发成本、损耗率和价格敏感度**。

8.3 差分进化求解

差分进化染色体包含 61 个选品优先级、61 个价格基因和 1 个品种数基因。品种数基因映射到 27—33；先在每个品类选择优先级最高的一个单品，保证六品类均有代表，再按优先级补足指定数量。价格基因线性映射到 $[p_i^L, p_i^U]$ ，补货量按上式恢复。算法采用 best/1/bin 策略，最大迭代 140 代、种群倍数 6、变异因子随机取 $[0.5, 1.0]$ 、交叉概率 0.85、随机种子 20230818，最后对连续变量执行边界内局部精修。

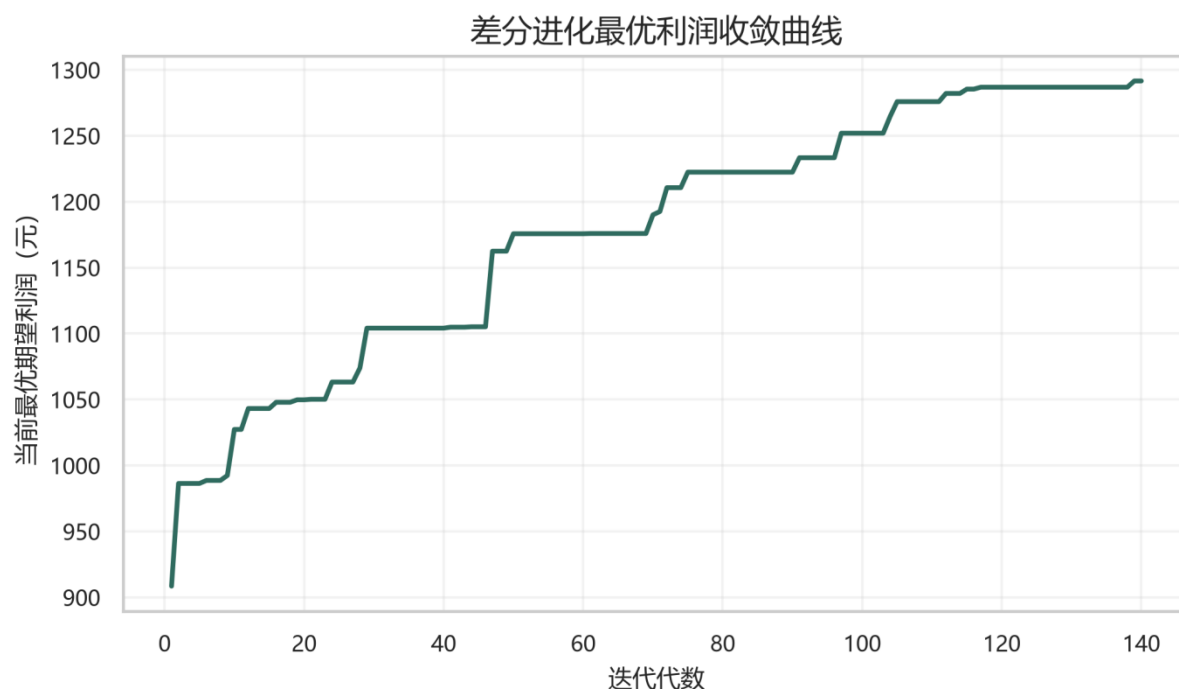


图 8 差分进化种群阶段的最优利润收敛曲线

图 8 反映了问题三中差分进化算法的寻优过程。种群阶段的当前最优利润由初始约 908.50 元逐步上升至第 140 代的 1291.58 元，整体呈阶梯式增长。曲线出现水平区间，表示连续若干代没有发现更优解；随后发生跳升，说明种群的变异和交叉操作搜索到了新的优质区域。

8.47 月 1 日单品策略

最终选择 33 种单品，满足 27—33 种约束且每种补货量不少于 2.5kg；总补货量 429.06kg，预计销量 392.13kg，经局部精修后的模型期望利润 1414.51 元。算法达到预设 140 代上限，种群阶段最优值在末期仍有小幅改进，因此该解应称为稳定可行近似解，而非数学上的全局最优证明。

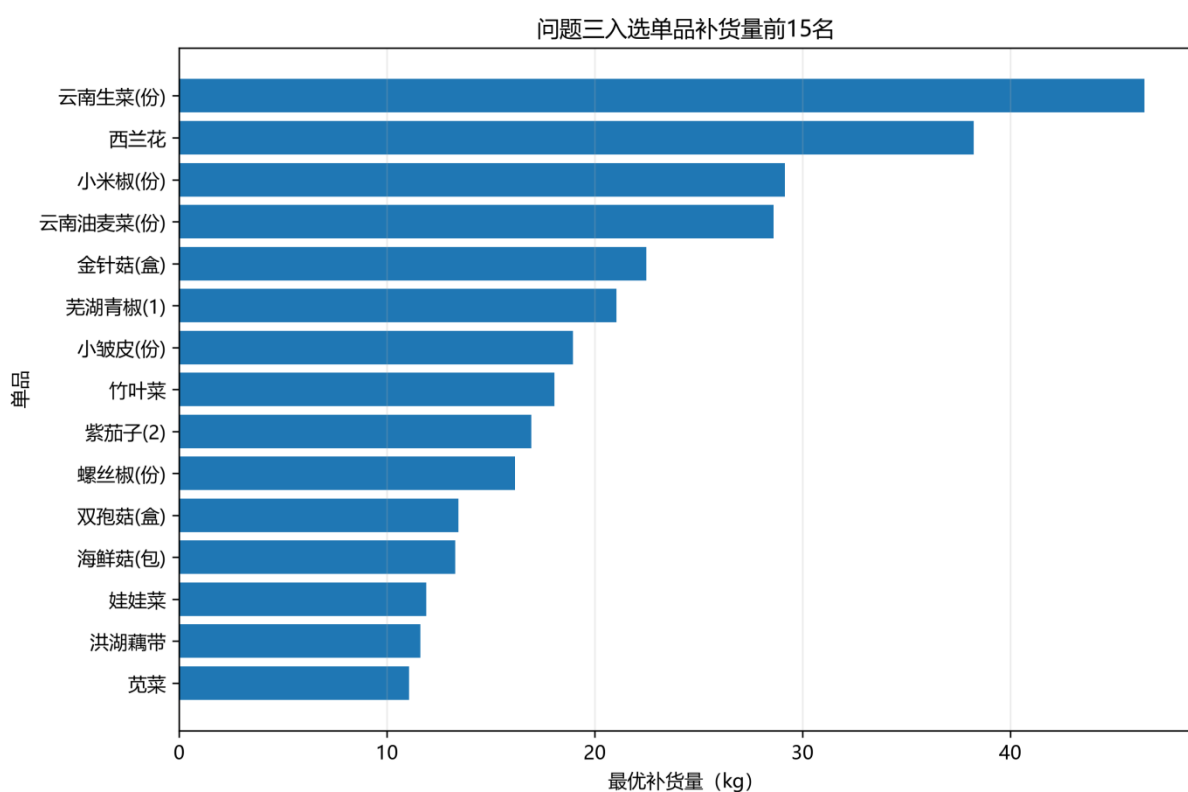
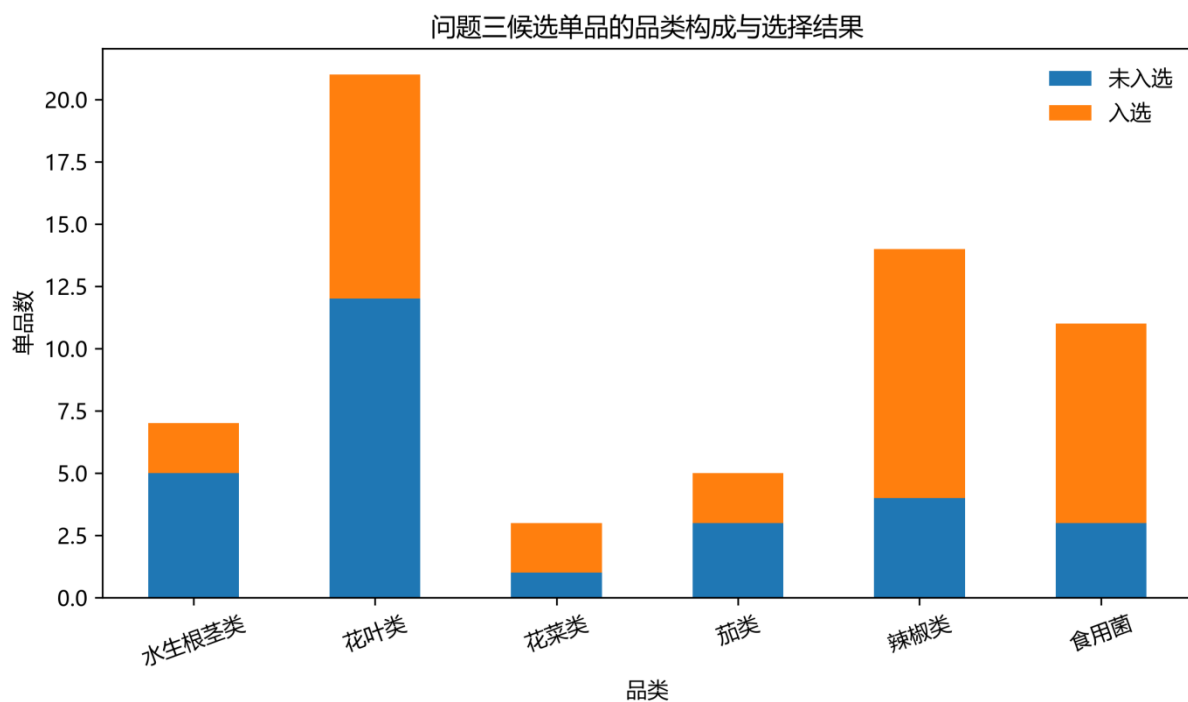


图 9, 10 61 种候选的选择构成及入选单品补货

图 9, 10 展示了 61 种候选单品在六个品类中的入选情况, 最终选择 33 种, 其中辣椒类 10 种、花叶类 9 种、食用菌 8 种, 花菜类、茄类和水生根茎类各 2 种。虽然选择结

果更集中于需求较大、利润贡献较高的品类，但六个品类均至少保留一种商品，满足品类覆盖约束。显示入选单品中补货量最大的 15 种商品。云南生菜（份）的补货量最高，约 46.45kg；其次为西兰花和小米椒（份），分别约 38.25kg 和 29.14kg。全部入选单品的总补货量为 429.06kg，预计销量 392.13kg，模型期望利润 1414.51 元。西兰花虽然补货量不是最高，但预计利润约 290.76 元，为单品利润贡献最高者，说明选品不能只依据销量，还需要综合售价、批发成本、损耗率和价格敏感度。

单品	品类	批发成本	建议售价	补货量/kg	预计利润/元
洪湖藕带	水生根茎类	18.00	27.05	11.62	29.55
红莲藕带	水生根茎类	5.60	11.33	2.50	9.01
竹叶菜	花叶类	2.32	6.80	18.05	64.24
云南生菜 (份)	花叶类	3.60	5.27	46.45	54.16
云南油麦菜 (份)	花叶类	2.86	4.90	28.61	45.09
娃娃菜	花叶类	4.73	7.98	11.88	36.23
红薯尖	花叶类	3.20	8.83	7.32	35.75
甜白菜	花叶类	4.98	9.12	9.65	31.69
木耳菜	花叶类	3.21	7.76	6.55	25.91
小青菜(1)	花叶类	2.83	6.36	8.45	24.24
苋菜	花叶类	2.32	4.97	11.06	19.09
西兰花	花菜类	7.82	17.00	38.25	290.76
枝江青梗散 花	花菜类	9.46	15.46	8.31	37.74
紫茄子(2)	茄类	3.75	7.40	16.95	54.28
长线茄	茄类	6.98	15.41	6.27	46.20
小米椒(份)	辣椒类	2.14	7.52	29.14	136.24
红椒(1)	辣椒类	10.79	19.88	7.34	49.53

单品	品类	批发成本	建议售价	补货量/kg	预计利润/元
芜湖青椒(1)	辣椒类	3.38	5.59	21.04	39.77
青线椒	辣椒类	6.92	16.69	4.60	38.93
小皱皮(份)	辣椒类	1.54	3.35	18.94	28.25
螺丝椒(份)	辣椒类	3.28	5.36	16.15	25.45
螺丝椒	辣椒类	7.52	11.03	10.57	25.25
姜蒜小米椒 组合装(小 份)	辣椒类	2.44	5.37	9.97	24.20
七彩椒(2)	辣椒类	12.21	22.59	2.50	18.12
红椒(2)	辣椒类	12.90	21.24	2.68	17.01
金针菇(1)	食用菌	4.02	9.53	7.94	41.22
西峡花菇(1)	食用菌	15.60	24.95	5.45	36.24
双孢菇(盒)	食用菌	3.40	6.10	13.43	36.13
金针菇(盒)	食用菌	1.45	2.89	22.49	32.12
杏鲍菇(1)	食用菌	5.50	12.16	3.89	23.51
海鲜菇(包)	食用菌	1.95	3.25	13.28	17.26
平菇	食用菌	4.59	9.27	4.73	17.02
虫草花(份)	食用菌	2.66	4.51	3.01	4.30

从品类覆盖看，33种结果包含全部六个品类；西兰花、小米椒(份)、竹叶菜、紫茄子(2)等单品贡献较高。最低陈列量在七彩椒(2)和红莲藕带等低需求单品上发生绑定，说明2.5kg陈列约束确实影响了组合利润和选品结果。

9 问题四：竞品价格与竞争反应函数

9.1 数据缺口与采集对象

附件没有商超地理位置，也没有竞争对手价格，所以实施时应以商超真实服务半径为范围，选择附近菜市场 and 主流生鲜平台，按同一单品或可比品质单品一同一日期和时段形成竞品面板。

由于题目附件没有提供竞争对手价格、市场份额和客户流失数据，本文暂时无法估计竞争反应系数、相对价格系数、本店价格敏感度和交叉价格敏感度。因此，问题四中的竞争反应方程、Logit 市场份额模型和客户流失率模型均属于后续数据采集后的扩展模型，不报告虚构的参数估计结果。待获得真实竞品数据后，若估计得到 $p > 0$ 说明竞争对手存在跟随本店调价的行为；本店价格相对竞争对手越高，客户选择本店的概率越低；竞争对手提价会使部分顾客转向本店。只有当这些系数在不同月份和不同价格区间内保持稳定时，才能将竞争模型接入自动定价系统。

数据类别	具体字段	作用
竞品价格	市场/平台、单品、标价、券后价、单位、时间戳	构造相对价格与交叉价格变量
可得性	是否有货、缺货时段、可售规格	区分价格替代与竞品缺货
交易条件	配送费、起送价、会员折扣、满减	计算消费者实际到手价格
品质匹配	产地、规格、包装、品级、新鲜度	防止把品质差异误判为价格效应
本店状态	本店售价、促销、库存、缺货、陈列位置	与竞品数据同步匹配
客户行为	客流、转化率、购物篮、会员复购	直接观察客户流失和替代去向

9.2 竞争对手调价反应方程

为分析竞争对手是否跟随本店调价，本文构建带滞后项的动态价格反应模型，计算公式如下：

$$\ln p_{it}^c = \alpha_i + \rho \ln p_{i,t-1} + \phi \ln p_{i,t-1}^c + \gamma^T Z_t + u_{it},$$

9.3 顾客份额与替代效应

为比较本店和竞争对手的实际价格水平，本文采用对数相对价格指标，计算公式如下：

$$g_{it} = \ln \frac{p_{it}}{p_{it}^c}.$$

为分析相对价格对顾客选择本店概率的影响，本文采用二项 Logit 模型，计算公式如下：

$$\ln \frac{s_{it}}{1-s_{it}} = \mu_i + \eta g_{it} + \delta^T X_{it} + \varepsilon_{it}, \quad \eta < 0,$$

在只能获得本店销量而无法获得完整市场份额时，本文采用恒定价格弹性模型修正问题二的基准需求，计算公式如下：

$$D_{it} = B_{it} \exp \left[-\eta_o \ln \frac{p_{it}}{p_i^o} + \eta_c \ln \frac{p_{it}^c}{p_i^{c,o}} \right],$$

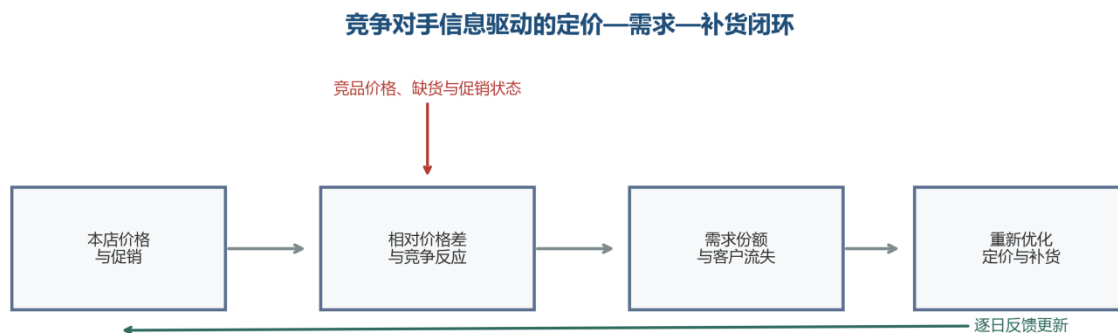
为量化本店提价后顾客转向竞争对手的比例，本文采用需求变化比例计算客户流失率，计算公式如下：

$$L_{it} = 1 - \frac{D_{it}(p', p^c)}{D_{it}(p, p^c)}.$$

图 10 竞品价格采集、份额反应与决策反馈闭环

9.4 识别、验证与接入优化

为减少新的内生性，应优先使用本店分时小幅随机调价、平台券随机发放或竞品突发缺货作为准实验；若无法实施试验，可用批发成本冲击作工具变量，并加入单品固定效应和日期固定效应。估计后将问题二需求替换为 $D_{it}(p, \hat{p}^c)$ ，把未来竞品价格由反应方程预测，再重新运行模拟退火；问题三的单品需求同样加入竞品相对价格，从而避免本店提价时低估客户外流



。

验证采用滚动时间留出，比较加入竞品变量前后的销量 MAE、客户份额误差、实际毛利和缺货率。若相对价格系数在不同月份、不同竞争渠道和不同价格区间中符号不稳定，则不允许模型自动提价，只保留监测功能。

十、模型检验与结果讨论

10.1 统计检验与时间隔离

问题一的 K-S 检验采用 5%显著性水平，Pearson 相关同时报告方向、强度与显著性。问题二只使用 2023 年 6 月 30 日及以前的数据预测 7 月 1 日至 7 日，并以最后 84 天进行时间留出检验；问题三的候选、价格与成本只使用 7 月 1 日前信息，避免未来信息泄漏。

10.2 算法可复现性

模拟退火和差分进化均固定随机种子 20230818。模拟退火对每个品类日执行 4 次独立重启并保留最优解；差分进化用可行性修复保证单品数、最低陈列和品类覆盖约束。所有决策表由同一 Python 程序直接导出，论文参数与代码一致。

10.3 结果边界

第二问的局部 R^2 较低且四类的预测 MAE 未优于 7 日朴素法，说明历史售价并不能充分解释销量波动；同时最优价格多位于 P90 边界，必须把 12312.62 元理解为模型内部期望利润。第三问采用价格记录代理可采购状态，61 种候选不等于 7 月 1 日真实到货清单；1414.51 元同样依赖需求分配、线性价格关系和零残值假设。

十一、模型评价

11.1 优点

第一问严格按完整日历、低销量筛选和箱线缩尾统一统计口径，分布检验与相关分析层次清晰；

归一化销量加权价格指数减少了品类内部结构变化造成的均价偏误；

Prophet 式加性分解和残差化 OLS 把趋势、季节、节假日与局部价格影响显式分开；

正价一折扣利润模型、模拟退火、MIQP 与差分进化从品类决策落实到单品清单；

第四问不泛泛罗列数据，而把竞品价格直接连接到客户流失、替代效应和前述优化模型。

11.2 不足

残差化只能控制可观测时间结构，不能完全解决价格与需求的同时性，因果识别仍需随机调价或有效工具变量；

题目未提供天气、库存、缺货、动态新鲜度和竞品面板，相关影响留在残差中；

单品需求由近期销量份额分配，未显式估计单品之间的交叉价格弹性；

差分进化在 140 代达到迭代上限，只能给出高质量近似解，不能证明全局最优；

利润结果是确定性条件期望，尚未传播需求、成本和损耗参数的不确定性。

11.3 推广

模型可推广到水果、肉类、烘焙和乳制品等短保商品。更换品类价格指数的商品篮子、损耗率和最低陈列约束后，仍可沿用加性需求分解—局部价格效应—分段销售利润—组合优化—竞争反馈的框架。

十二、结论

本文完成了从销售规律到经营执行的完整链条。问题一表明六品类存在显著周月周期且两两分布不同，重点单品之间同时存在正负相关。问题二构建归一化销量加权价格指数和 Prophet 式加性模型，六品类局部价格边际系数均为负，并由模拟退火形成未来 7 日品类决策。问题三从 61 种候选中选出 33 种，给出 7 月 1 日售价、补货量和预计利润。问题四进一步以竞品价格为核心构建调价反应和客户替代模型。结果可作为商超短期决策的量化基准，但价格边界、低局部解释度、采购可得性代理及竞品数据缺失均要求实际部署采用滚动更新与人工审核。

AI 使用说明

本文写作过程中使用 chatgpt 人工智能作为辅助工具，主要用于论文结构梳理、语言表达润色、程序语法检查和版式规范化。研究问题的理解、数据清洗规则、模型选择与推导、参数设定、结果计算、图表核验及结论形成均由参赛者独立完成并复核；人工智能未替代参赛者作出实质性建模判断。

参考文献

- [1]全国大学生数学建模竞赛组委会. 2023 年全国大学生数学建模竞赛 C 题：蔬菜类商品的自动定价与补货决策[Z]. 2023.
- [2]TaylorSJ, LethamB. Forecastingatscale[J]. TheAmericanStatistician, 2018, 72(1):37-45.
- [3]FrischR, WaughFV. Partialtimeregressionsascomparedwithindividualtrends[J]. Econometrica, 1933, 1(4):387-401.
- [4]LovellMC. Seasonaladjustmentofeconomic timeseriesandmultiple regressionanalysis[J]. JournaloftheAmericanStatisticalAssociation, 1963, 58(304):993-1010.
- [5]KirkpatrickS, GelattCD, VecchiMP. Optimizationbysimulatedannealing[J]. Science, 1983, 220(4598):671-680.
- [6]StornR, PriceK. Differentialevolution—
Asimpleandefficientheuristicforglobaloptimizationovercontinuousspaces[J]. JournalofGlobalOptimization, 1997, 11:341-359.
- [7]PetruzziNC, DadaM. Pricingandthenewsvendorproblem:Areviewwithextensions[J]. OperationsResearch, 1999, 47(2):183-194.
- [8]FanT, XuC, TaoF. Dynamicpricingandreplenishmentpolicyforfreshproduce[J]. Computers&IndustrialEngineering, 2020, 139:106127.
- [9]AnupindiR, DadaM, GuptaS. Estimationofconsumerdemandwithstock-outbasedsubstitution[J]. MarketingScience, 1998, 17(4):406-423.
- [10]MahajanS, vanRyzinG. Stockingretailassortmentsunderdynamicconsumersubstitution[J]. OperationsResearch, 2001, 49(3):334-351.
- [11]PearsonK. Notesonregressionandinheritanceinthecaseoftwoparents[J]. ProceedingsoftheRoyalSocietyofLondon, 1895, 58:240-242.

- [12] Smirnov N. Table for estimating the goodness of fit of empirical distributions [J]. The Annals of Mathematical Statistics, 1948, 19 (2) : 279–281.
- [13] Tukey JW. Exploratory Data Analysis [M]. Reading, MA: Addison–Wesley, 1977.
- [14] Greene WH. Econometric Analysis [M]. 8th ed. New York: Pearson, 2018.
- [15] Berry S, Levinsohn J, Pakes A. Automobile prices in market equilibrium [J]. Econometrica, 1995, 63 (4) : 841–890.

附录 A 算法代码与参数

A.1 问题一

补齐 1095 日历日并填 0；按累计销量第 25 百分位筛选；对保留单品正销量日计算 Q1、Q3 和 IQR 并缩尾；聚合到品类后生成周月柱状图、两两 K-S 检验、品类 Pearson 聚类及销量前 15 单品相关矩阵。

A.2 问题二模拟退火

对每个品类日初始化 (p, q, θ) ；在有界区间内按温度缩放的高斯邻域产生候选；若利润提高则接受，否则以 $\exp(\Delta\pi/T)$ 概率接受；每轮令 T 乘 0.997；5000 次后结束，独立重启 4 次并保留利润最大解。

A.3 问题三差分进化

初始化连续种群；将选品数量映射到 27—33；按优先级选择并执行六品类覆盖修复；将价格基因映射到边界，依据线性需求和损耗恢复最优补货；计算利润；执行 best/1 变异、二项交叉和贪婪选择；迭代 140 代后局部精修并输出 33 种清单。

附录

问题一

```
from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib.font_manager as font_manager
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage
from scipy.spatial.distance import squareform
from scipy.stats import ks_2samp, pearsonr, skew

# ===== 1. 全局参数 =====

START_DATE = pd.Timestamp("2020-07-01")
END_DATE = pd.Timestamp("2023-06-30")
FULL_CALENDAR = pd.date_range(START_DATE, END_DATE, freq="D")

CATEGORY_ORDER = [
    "花叶类",
    "花菜类",
    "水生根茎类",
    "茄类",
    "辣椒类",
    "食用菌",
]

CATEGORY_COLORS = {
    "花叶类": "#66C2A5",
    "花菜类": "#FC8D62",
    "水生根茎类": "#8DA0CB",
    "茄类": "#E78AC3",
    "辣椒类": "#A6D854",
    "食用菌": "#FFD92F",
}
```

```

# 箱线图异常值倍数；论文采用 1.5 倍四分位距。
IQR_MULTIPLIER = 1.5

# 品类相关聚类的截断距离；论文采用  $d=1-r$ ，截断值为 0.55。
CLUSTER_DISTANCE_THRESHOLD = 0.55

# 图片分辨率。
FIGURE_DPI = 300

# ===== 2. 通用函数 =====

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="生成 2023 年 C 题问题一全部图表")
    parser.add_argument(
        "--data-dir",
        type=Path,
        default=Path("."),
        help="附件 1、附件 2 所在文件夹，默认当前文件夹",
    )
    parser.add_argument(
        "--output-dir",
        type=Path,
        default=Path(r"D:\code\problem1_result"),
        help=r"图表输出文件夹，默认 D:\code\problem1_result",
    )
    parser.add_argument(
        "--show",
        action="store_true",
        help="保存图片后同时弹出图形窗口",
    )
    return parser.parse_args()

def find_attachment(data_dir: Path, attachment_number: int) -> Path:
    """自动兼容“附件 1.xlsx”“附件 1(1).xlsx”等文件名。"""
    candidates = sorted(
        data_dir.glob(f"附件{attachment_number}*.xlsx"),
        key=lambda p: (len(p.name), p.name),
    )
    if not candidates:
        raise FileNotFoundError(
            f"在 {data_dir.resolve()} 中没有找到附件{attachment_number}*.xlsx"
        )

```

```

    return candidates[0]

def configure_chinese_font() -> str | None:
    """优先选择常见中文字体；找不到时仍可运行，但中文可能显示为方框。"""
    preferred_fonts = [
        "Microsoft YaHei",
        "SimHei",
        "SimSun",
        "Noto Sans CJK SC",
        "Source Han Sans SC",
        "WenQuanYi Micro Hei",
        "Arial Unicode MS",
    ]
    installed = {f.name for f in font_manager.fontManager.ttflist}
    selected = next((name for name in preferred_fonts if name in installed), None)
    if selected:
        plt.rcParams["font.sans-serif"] = [selected, "DejaVu Sans"]
    else:
        print("警告：未检测到常见中文字体，请安装微软雅黑、黑体或思源黑体。")
        plt.rcParams["font.sans-serif"] = ["DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False
    return selected

def set_plot_style() -> None:
    sns.set_theme(style="whitegrid", context="notebook")
    configure_chinese_font()
    plt.rcParams.update(
        {
            "figure.dpi": 120,
            "savefig.dpi": FIGURE_DPI,
            "axes.titlesize": 13,
            "axes.labelsize": 10,
            "xtick.labelsize": 9,
            "ytick.labelsize": 9,
            "legend.fontsize": 9,
        }
    )

def save_figure(fig: plt.Figure, output_dir: Path, stem: str, show: bool) -> None:
    """同时保存 300 dpi PNG 和矢量 PDF。"""
    fig.tight_layout()

```

```

fig.savefig(output_dir / f"{stem}.png", dpi=FIGURE_DPI, bbox_inches="tight",
facecolor="white")
fig.savefig(output_dir / f"{stem}.pdf", bbox_inches="tight", facecolor="white")
if show:
    plt.show()
plt.close(fig)

def iqr_winsorize_positive(
    series: pd.Series,
    multiplier: float = IQR_MULTIPLIER,
) -> tuple[pd.Series, float, float, int]:
    """
    对一个单品的日销量做箱线图异常处理。

    只在正销量日计算四分位数，0 表示无销售，予以保留；
    异常值不删除，而是缩尾到箱线图上下须。
    """
    result = series.astype(float).copy()
    positive = result[result > 0]

    if len(positive) < 4:
        return result, np.nan, np.nan, 0

    q1 = positive.quantile(0.25)
    q3 = positive.quantile(0.75)
    iqr = q3 - q1
    lower = max(0.0, q1 - multiplier * iqr)
    upper = q3 + multiplier * iqr

    abnormal = (result > 0) & ((result < lower) | (result > upper))
    result.loc[(result > 0) & (result < lower)] = lower
    result.loc[result > upper] = upper

    return result, lower, upper, int(abnormal.sum())

def safe_pearson(x: pd.Series, y: pd.Series) -> tuple[float, float]:
    """避免常数序列导致 pearsonr 报错。"""
    if x.nunique() <= 1 or y.nunique() <= 1:
        return np.nan, np.nan
    result = pearsonr(x, y)
    return float(result.statistic), float(result.pvalue)

```

```

# ===== 3. 数据预处理 =====

def prepare_q1_data(data_dir: Path, output_dir: Path) -> dict[str, object]:
    attachment_1 = find_attachment(data_dir, 1)
    attachment_2 = find_attachment(data_dir, 2)

    print(f"读取商品信息: {attachment_1}")
    print(f"读取销售流水: {attachment_2}")

    items = pd.read_excel(
        attachment_1,
        dtype={"单品编码": str, "分类编码": str},
    )
    sales = pd.read_excel(
        attachment_2,
        dtype={"单品编码": str},
    )

    required_item_columns = {"单品编码", "单品名称", "分类名称"}
    required_sales_columns = {"销售日期", "单品编码", "销量(千克)", "销售类型"}
    if not required_item_columns.issubset(items.columns):
        raise ValueError(f"附件 1 缺少字段: {required_item_columns -
set(items.columns)}")
    if not required_sales_columns.issubset(sales.columns):
        raise ValueError(f"附件 2 缺少字段: {required_sales_columns -
set(sales.columns)}")

    items = items.drop_duplicates(subset=["单品编码"]).copy()
    sales["销售日期"] = pd.to_datetime(sales["销售日期"], errors="coerce")
    sales["销量(千克)"] = pd.to_numeric(sales["销量(千克)"],
errors="coerce").fillna(0.0)
    sales = sales[sales["销售日期"].between(START_DATE, END_DATE)].copy()

    # 若某份附件将退货量记为正数，这里转为负数；原附件已经是负数时不会重复改号。
    return_mask = sales["销售类型"].astype(str).str.contains("退货", na=False)
    positive_return_mask = return_mask & (sales["销量(千克)"] > 0)
    sales.loc[positive_return_mask, "销量(千克)"] *= -1

    # 逐笔流水聚合为“日期-单品”净销量。
    observed_daily = (
        sales.groupby(["销售日期", "单品编码"], as_index=False)["销量(千克)"]

```

```

        .sum()
        .rename(columns={"销量(千克)": "日净销量"})
    )
    observed_daily = observed_daily.merge(
        items[["单品编码", "单品名称", "分类名称"]],
        on="单品编码",
        how="left",
        validate="many_to_one",
    )

    # 先计算 251 个登记单品的三年累计净销量，未出现的单品自动补 0。
    registered_items = items[["单品编码", "单品名称", "分类名称"]].copy()
    item_total = (
        observed_daily.groupby(["单品编码", "单品名称", "分类名称"], as_index=False)["
日净销量"]
        .sum()
        .rename(columns={"日净销量": "三年累计净销量"})
    )
    item_total = registered_items.merge(
        item_total,
        on=["单品编码", "单品名称", "分类名称"],
        how="left",
    )
    item_total["三年累计净销量"] = item_total["三年累计净销量"].fillna(0.0)

    # “删除三年总销量小于 25%的商品”解释为删除累计销量低于第 25 百分位的单品。
    q25_threshold = float(item_total["三年累计净销量"].quantile(0.25))
    item_total["是否保留"] = item_total["三年累计净销量"] >= q25_threshold
    kept_codes = item_total.loc[item_total["是否保留"], "单品编码"].tolist()

    # 构造 1095 天×保留单品的完整面板，无销售记录用 0 替代。
    full_index = pd.MultiIndex.from_product(
        [FULL_CALENDAR, kept_codes],
        names=["销售日期", "单品编码"],
    )
    full_daily = (
        observed_daily.set_index(["销售日期", "单品编码"])[["日净销量"]]
        .reindex(full_index, fill_value=0.0)
        .reset_index()
        .merge(
            items[["单品编码", "单品名称", "分类名称"]],
            on="单品编码",
            how="left",

```

```

        validate="many_to_one",
    )
)

# 日净销量为负时按 0 处理，避免把集中退货解释为负需求。
full_daily["日净销量"] = full_daily["日净销量"].clip(lower=0.0)
zero_cells = int((full_daily["日净销量"] == 0).sum())

# 按单品执行箱线图缩尾。
cleaned_groups: list[pd.DataFrame] = []
iqr_records: list[dict[str, object]] = []
for item_code, group in full_daily.groupby("单品编码", sort=False):
    cleaned_sales, lower, upper, abnormal_count = iqr_winsorize_positive(group["
日净销量"])
    temp = group.copy()
    temp["清洗后日销量"] = cleaned_sales.to_numpy()
    cleaned_groups.append(temp)
    iqr_records.append(
        {
            "单品编码": item_code,
            "单品名称": group["单品名称"].iloc[0],
            "分类名称": group["分类名称"].iloc[0],
            "箱线图下须": lower,
            "箱线图上须": upper,
            "异常日期数": abnormal_count,
        }
    )

cleaned_daily = pd.concat(cleaned_groups, ignore_index=True)
iqr_table = pd.DataFrame(iqr_records)

# 聚合成六品类 1095 天日销量矩阵。
category_daily = (
    cleaned_daily.groupby(["销售日期", "分类名称"], as_index=False)["清洗后日销量"]
    .sum()
    .pivot(index="销售日期", columns="分类名称", values="清洗后日销量")
    .reindex(FULL_CALENDAR)
    .fillna(0.0)
    .reindex(columns=CATEGORY_ORDER)
)

# 描述性统计。
descriptive_rows: list[dict[str, float | str]] = []
for category in CATEGORY_ORDER:

```

```

values = category_daily[category]
descriptive_rows.append(
    {
        "分类名称": category,
        "均值": values.mean(),
        "标准差": values.std(ddof=1),
        "最小值": values.min(),
        "25%分位数": values.quantile(0.25),
        "中位数": values.median(),
        "75%分位数": values.quantile(0.75),
        "最大值": values.max(),
        "变异系数": values.std(ddof=1) / values.mean(),
        "偏度": skew(values, bias=False),
    }
)
descriptive = pd.DataFrame(descriptive_rows)

# 星期和月份平均销量。
weekday_average = category_daily.groupby(category_daily.index.dayofweek).mean()
weekday_average.index = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]

month_average = category_daily.groupby(category_daily.index.month).mean()
month_average.index = [f"{month}月" for month in range(1, 13)]

# 六品类两两 K-S 检验。
ks_statistic = pd.DataFrame(
    np.zeros((len(CATEGORY_ORDER), len(CATEGORY_ORDER))),
    index=CATEGORY_ORDER,
    columns=CATEGORY_ORDER,
)
ks_pvalue = pd.DataFrame(
    np.ones((len(CATEGORY_ORDER), len(CATEGORY_ORDER))),
    index=CATEGORY_ORDER,
    columns=CATEGORY_ORDER,
)
ks_records: list[dict[str, object]] = []
for i, category_a in enumerate(CATEGORY_ORDER):
    for j in range(i + 1, len(CATEGORY_ORDER)):
        category_b = CATEGORY_ORDER[j]
        result = ks_2samp(
            category_daily[category_a],
            category_daily[category_b],
            alternative="two-sided",

```



```

        method="auto",
    )
    ks_statistic.loc[category_a, category_b] = result.statistic
    ks_statistic.loc[category_b, category_a] = result.statistic
    ks_pvalue.loc[category_a, category_b] = result.pvalue
    ks_pvalue.loc[category_b, category_a] = result.pvalue
    ks_records.append(
        {
            "品类 A": category_a,
            "品类 B": category_b,
            "K-S 统计量": float(result.statistic),
            "p 值": float(result.pvalue),
            "5%水平结论": "拒绝同分布" if result.pvalue < 0.05 else "未拒绝同分
布",
        }
    )
ks_pairwise = pd.DataFrame(ks_records)

# 品类 Pearson 相关矩阵与系统聚类。
category_correlation = category_daily.corr(method="pearson")
category_distance = (1.0 - category_correlation).clip(lower=0.0)

# 复制为独立、可写的 NumPy 数组，再将对角线严格置 0。
# 这样可避免部分 pandas / NumPy 版本下视图为只读而导致报错。
dist_mat = category_distance.to_numpy(dtype=float, copy=True)
np.fill_diagonal(dist_mat, 0.0)
category_distance = pd.DataFrame(
    dist_mat,
    index=category_distance.index,
    columns=category_distance.columns,
)

category_linkage = linkage(
    squareform(category_distance.to_numpy(), checks=False),
    method="average",
)
cluster_number = fcluster(
    category_linkage,
    t=CLUSTER_DISTANCE_THRESHOLD,
    criterion="distance",
)
category_cluster = pd.DataFrame(
    {"分类名称": CATEGORY_ORDER, "聚类编号": cluster_number}

```

```

)

# 清洗后的三年累计销量前 15 单品。
cleaned_item_total = (
    cleaned_daily.groupby(["单品编码", "单品名称", "分类名称"], as_index=False)["清洗后日销量"]
    .sum()
    .sort_values("清洗后日销量", ascending=False)
    .rename(columns={"清洗后日销量": "三年累计清洗后销量"})
)

top15_items = cleaned_item_total.head(15).copy()
top15_names = top15_items["单品名称"].tolist()
top15_codes = top15_items["单品编码"].tolist()
top15_items.insert(0, "图中编号", [f"I{i}" for i in range(1, 16)])

top15_daily = (
    cleaned_daily[cleaned_daily["单品编码"].isin(top15_codes)]
    .pivot(index="销售日期", columns="单品名称", values="清洗后日销量")
    .reindex(FULL_CALENDAR)
    .fillna(0.0)
    .reindex(columns=top15_names)
)

top15_correlation = top15_daily.corr(method="pearson")
top15_pvalue = pd.DataFrame(
    np.ones_like(top15_correlation),
    index=top15_correlation.index,
    columns=top15_correlation.columns,
)

item_pair_records: list[dict[str, object]] = []
for i, item_a in enumerate(top15_names):
    for j in range(i + 1, len(top15_names)):
        item_b = top15_names[j]
        correlation, pvalue = safe_pearson(top15_daily[item_a],
top15_daily[item_b])
        top15_pvalue.loc[item_a, item_b] = pvalue
        top15_pvalue.loc[item_b, item_a] = pvalue
        item_pair_records.append(
            {
                "单品 A": item_a,
                "单品 B": item_b,
                "Pearson 相关系数": correlation,
                "p 值": pvalue,
                "相关方向": "正相关" if correlation > 0 else "负相关",
            }
        )

```

```

    }
)
item_pairs = pd.DataFrame(item_pair_records).sort_values(
    "Pearson 相关系数", ascending=False
)

# 导出绘图和论文制表所需数据。
item_total.to_csv(output_dir / "表 1_单品三年累计销量及筛选结果.csv", index=False,
encoding="utf-8-sig")
iqr_table.to_csv(output_dir / "表 2_各单品箱线图边界.csv", index=False,
encoding="utf-8-sig")
descriptive.to_csv(output_dir / "表 3_六品类描述性统计.csv", index=False,
encoding="utf-8-sig")
weekday_average.to_csv(output_dir / "表 4_星期平均销量.csv", encoding="utf-8-sig")
month_average.to_csv(output_dir / "表 5_月份平均销量.csv", encoding="utf-8-sig")
ks_pairwise.to_csv(output_dir / "表 6_六品类两两 KS 检验.csv", index=False,
encoding="utf-8-sig")
ks_statistic.to_csv(output_dir / "表 7_KS 统计量矩阵.csv", encoding="utf-8-sig")
ks_pvalue.to_csv(output_dir / "表 8_KS 检验 p 值矩阵.csv", encoding="utf-8-sig")
category_correlation.to_csv(output_dir / "表 9_品类 Pearson 相关矩阵.csv",
encoding="utf-8-sig")
category_cluster.to_csv(output_dir / "表 10_品类相关性聚类结果.csv", index=False,
encoding="utf-8-sig")
top15_items.to_csv(output_dir / "表 11_累计销量前 15 单品.csv", index=False,
encoding="utf-8-sig")
top15_correlation.to_csv(output_dir / "表 12_前 15 单品 Pearson 相关矩阵.csv",
encoding="utf-8-sig")
top15_pvalue.to_csv(output_dir / "表 13_前 15 单品 Pearson 检验 p 值矩阵.csv",
encoding="utf-8-sig")
item_pairs.to_csv(output_dir / "表 14_前 15 单品相关对排序.csv", index=False,
encoding="utf-8-sig")

summary = {
    "登记单品数": int(len(item_total)),
    "累计销量第 25 百分位门槛_kg": q25_threshold,
    "保留单品数": int(item_total["是否保留"].sum()),
    "删除单品数": int((~item_total["是否保留"]).sum()),
    "完整日历天数": int(len(FULL_CALENDAR)),
    "补零后的零销量单元数": zero_cells,
    "箱线法修正的单品日期数": int(iqr_table["异常日期数"].sum()),
    "KS 检验拒绝同分布的品类对数": int((ks_pairwise["p 值"] < 0.05).sum()),
    "KS 检验品类对总数": int(len(ks_pairwise)),
    "前 15 单品正相关对数": int((item_pairs["Pearson 相关系数"] > 0).sum()),

```

```

        "前 15 单品负相关对数": int((item_pairs["Pearson 相关系数"] < 0).sum()),
        "前 15 单品 5%水平显著相关对数": int((item_pairs["p 值"] < 0.05).sum()),
    }
    (output_dir / "问题一处理结果摘要.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    return {
        "summary": summary,
        "category_daily": category_daily,
        "weekday_average": weekday_average,
        "month_average": month_average,
        "ks_statistic": ks_statistic,
        "category_correlation": category_correlation,
        "category_linkage": category_linkage,
        "top15_items": top15_items,
        "top15_correlation": top15_correlation,
    }

# ===== 4. 图表绘制 =====

def plot_weekday_sales(
    weekday_average: pd.DataFrame,
    output_dir: Path,
    show: bool,
) -> None:
    fig, axes = plt.subplots(2, 3, figsize=(12, 7), sharex=True)
    for ax, category in zip(axes.flat, CATEGORY_ORDER):
        values = weekday_average[category]
        bars = ax.bar(
            values.index,
            values.to_numpy(),
            color=CATEGORY_COLORS[category],
            width=0.72,
            edgecolor="white",
            linewidth=0.5,
        )
        ax.set_title(category, fontweight="bold")
        ax.set_ylabel("平均日销量 (kg) ")
        ax.tick_params(axis="x", rotation=0)
        ax.grid(axis="y", linestyle="--", alpha=0.35)

```

```

        max_index = int(np.argmax(values.to_numpy()))
        max_bar = bars[max_index]
        ax.text(
            max_bar.get_x() + max_bar.get_width() / 2,
            max_bar.get_height(),
            f"{max_bar.get_height():.1f}",
            ha="center",
            va="bottom",
            fontsize=8,
        )
    fig.suptitle("六个蔬菜品类的星期平均销量", fontsize=16, fontweight="bold", y=1.02)
    save_figure(fig, output_dir, "图 2_六品类星期平均销量柱状图", show)

def plot_month_sales(
    month_average: pd.DataFrame,
    output_dir: Path,
    show: bool,
) -> None:
    fig, axes = plt.subplots(2, 3, figsize=(12, 7), sharex=True)
    for ax, category in zip(axes.flat, CATEGORY_ORDER):
        values = month_average[category]
        bars = ax.bar(
            values.index,
            values.to_numpy(),
            color=CATEGORY_COLORS[category],
            width=0.72,
            edgecolor="white",
            linewidth=0.5,
        )
        ax.set_title(category, fontweight="bold")
        ax.set_ylabel("平均日销量 (kg) ")
        ax.tick_params(axis="x", rotation=0)
        ax.grid(axis="y", linestyle="--", alpha=0.35)
        max_index = int(np.argmax(values.to_numpy()))
        max_bar = bars[max_index]
        ax.text(
            max_bar.get_x() + max_bar.get_width() / 2,
            max_bar.get_height(),
            f"{max_bar.get_height():.1f}",
            ha="center",
            va="bottom",
            fontsize=8,
        )

```

```

    )
    fig.suptitle("六个蔬菜品类的月份平均销量", fontsize=16, fontweight="bold", y=1.02)
    save_figure(fig, output_dir, "图 3_六品类月份平均销量柱状图", show)

def plot_ks_and_category_cluster(
    ks_statistic: pd.DataFrame,
    category_linkage: np.ndarray,
    output_dir: Path,
    show: bool,
) -> None:
    fig, axes = plt.subplots(
        1,
        2,
        figsize=(13, 5.4),
        gridspec_kw={"width_ratios": [1.05, 1.0]},
    )

    sns.heatmap(
        ks_statistic,
        annot=True,
        fmt=".2f",
        cmap="YlOrRd",
        vmin=0,
        vmax=1,
        square=True,
        linewidths=0.5,
        linecolor="white",
        cbar_kws={"label": "K-S 统计量", "shrink": 0.82},
        ax=axes[0],
    )

    axes[0].set_title(" (a) 六品类两两 K-S 统计量", fontweight="bold")
    axes[0].set_xlabel("")
    axes[0].set_ylabel("")
    axes[0].tick_params(axis="x", rotation=35)
    axes[0].tick_params(axis="y", rotation=0)

    dendrogram(
        category_linkage,
        labels=CATEGORY_ORDER,
        orientation="right",
        color_threshold=CLUSTER_DISTANCE_THRESHOLD,
        above_threshold_color="#4C78A8",
    )

```

```

        ax=axes[1],
    )
    axes[1].axvline(
        CLUSTER_DISTANCE_THRESHOLD,
        color="#C44E52",
        linestyle="--",
        linewidth=1.2,
        label=f"截断距离={CLUSTER_DISTANCE_THRESHOLD}",
    )
    axes[1].set_title("(b) 品类 Pearson 相关系统聚类", fontweight="bold")
    axes[1].set_xlabel("距离  $d = 1 - r$ ")
    axes[1].set_ylabel("")
    axes[1].legend(frameon=False, loc="lower right")
    axes[1].grid(axis="x", linestyle="--", alpha=0.35)

    fig.suptitle("品类销量分布差异与相关性分类", fontsize=16, fontweight="bold",
y=1.02)
    save_figure(fig, output_dir, "图 4_品类 KS 统计量与 Pearson 相关系统聚类", show)

def plot_top15_item_correlation(
    top15_items: pd.DataFrame,
    top15_correlation: pd.DataFrame,
    output_dir: Path,
    show: bool,
) -> None:
    labels = top15_items["图中编号"].tolist()

    fig, ax = plt.subplots(figsize=(9, 8))
    sns.heatmap(
        top15_correlation,
        cmap="RdBu_r",
        center=0,
        vmin=-1,
        vmax=1,
        square=True,
        xticklabels=labels,
        yticklabels=labels,
        linewidths=0.25,
        linecolor="white",
        cbar_kws={"label": "Pearson 相关系数", "shrink": 0.82},
        ax=ax,
    )

```

```

    ax.set_title("累计销量前 15 单品的 Pearson 相关系数", fontsize=15,
fontweight="bold", pad=14)
    ax.set_xlabel("单品编号 (对应表 11)")
    ax.set_ylabel("单品编号 (对应表 11)")
    ax.tick_params(axis="x", rotation=0)
    ax.tick_params(axis="y", rotation=0)
    save_figure(fig, output_dir, "图 5_销量前 15 单品 Pearson 相关系数热力图", show)

# ===== 5. 主程序 =====

def main() -> None:
    args = parse_args()
    data_dir = args.data_dir.expanduser().resolve()
    output_dir = args.output_dir.expanduser().resolve()
    output_dir.mkdir(parents=True, exist_ok=True)

    set_plot_style()
    results = prepare_q1_data(data_dir, output_dir)

    plot_weekday_sales(results["weekday_average"], output_dir, args.show)
    plot_month_sales(results["month_average"], output_dir, args.show)
    plot_ks_and_category_cluster(
        results["ks_statistic"],
        results["category_linkage"],
        output_dir,
        args.show,
    )
    plot_top15_item_correlation(
        results["top15_items"],
        results["top15_correlation"],
        output_dir,
        args.show,
    )

    print("\n 问题一全部图表已生成: ")
    print(output_dir)
    print("\n 关键结果: ")
    for key, value in results["summary"].items():
        print(f" {key}: {value}")

if __name__ == "__main__":
    main()

```


问题二

2023 年全国大学生数学建模竞赛 C 题—问题二独立建模代码

完成内容：

1. 读取附件 1--4，区分销售与退货，并聚合为“日期--单品”数据；
2. 按品类构造归一化销量加权售价指数和批发成本指数；
3. 使用 Prophet 风格加性结构（分段趋势 + Fourier 周/年周期 + 节假日）分解销量，并使用 FWL 残差化 + 局部 OLS 估计价格边际系数；
4. 使用最后 84 天做时间留出检验，并与 7 日季节朴素法比较；
5. 构建“正价销售--剩余商品折扣销售”利润模型；
6. 使用模拟退火求解 2023-07-01 至 2023-07-07 六个品类的最优价格指数、补货量和折价系数。

说明：

这里直接构造 Prophet 风格的设计矩阵，不依赖 prophet/fbprophet 包，可以减少不同 Python 环境中的安装问题。

安装依赖：

```
pip install pandas numpy scipy scikit-learn openpyxl
```

附件要求：

数据目录中放置附件 1.xlsx、附件 2.xlsx、附件 3.xlsx、附件 4.xlsx。
文件名为“附件 1(1).xlsx”等形式也能自动识别。

Windows 运行示例：

```
python 2023C 题_问题二建模代码.py ^  
--data-dir "D:/code/problem2_data" ^  
--output-dir "D:/code/problem2_result"
```

若四个附件与本代码位于同一目录，可直接运行：

```
python 2023C 题_问题二建模代码.py
```

"""

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import math
```

```
import os
```

```

from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error

# ===== 参数设置 =====

SEED = 20230818
HIST_START = pd.Timestamp("2020-07-01")
HIST_END = pd.Timestamp("2023-06-30")
FULL_CALENDAR = pd.date_range(HIST_START, HIST_END, freq="D")
FUTURE_DATES = pd.date_range("2023-07-01", "2023-07-07", freq="D")
CATEGORY_ORDER = ["花叶类", "花菜类", "水生根茎类", "茄类", "辣椒类", "食用菌"]

SCRIPT_DIR = Path(__file__).resolve().parent
DEFAULT_OUTPUT_DIR = (
    Path(r"D:\code\problem2_result")
    if os.name == "nt"
    else SCRIPT_DIR / "problem2_result"
)

# 研究期内的法定节假日区间。题目没有商超所在地，因此不引入天气数据。
HOLIDAY_INTERVALS = [
    ("2020-10-01", "2020-10-08"),
    ("2021-01-01", "2021-01-03"),
    ("2021-02-11", "2021-02-17"),
    ("2021-04-03", "2021-04-05"),
    ("2021-05-01", "2021-05-05"),
    ("2021-06-12", "2021-06-14"),
    ("2021-09-19", "2021-09-21"),
    ("2021-10-01", "2021-10-07"),
    ("2022-01-01", "2022-01-03"),
    ("2022-01-31", "2022-02-06"),
    ("2022-04-03", "2022-04-05"),
    ("2022-04-30", "2022-05-04"),
    ("2022-06-03", "2022-06-05"),
    ("2022-09-10", "2022-09-12"),
    ("2022-10-01", "2022-10-07"),

```

```

        ("2022-12-31", "2023-01-02"),
        ("2023-01-21", "2023-01-27"),
        ("2023-04-05", "2023-04-05"),
        ("2023-04-29", "2023-05-03"),
        ("2023-06-22", "2023-06-24"),
    ]

@dataclass
class Settings:
    data_dir: Path
    output_dir: Path
    sa_iterations: int = 5000
    sa_restarts: int = 4
    ridge_alpha: float = 10.0

def parse_args() -> Settings:
    parser = argparse.ArgumentParser(description="运行 2023 年 C 题问题二建模计算")
    parser.add_argument(
        "--data-dir",
        type=Path,
        default=SCRIPT_DIR,
        help="四个附件所在目录；默认是代码所在目录",
    )
    parser.add_argument(
        "--output-dir",
        type=Path,
        default=DEFAULT_OUTPUT_DIR,
        help=r"结果目录；Windows 默认 D:\code\problem2_result",
    )
    parser.add_argument("--sa-iterations", type=int, default=5000, help="每次模拟退火迭代数")
    parser.add_argument("--sa-restarts", type=int, default=4, help="每个品类日的模拟退火重启次数")
    parser.add_argument("--ridge-alpha", type=float, default=10.0, help="加性时间模型岭回归系数")
    args = parser.parse_args()
    if args.sa_iterations <= 0 or args.sa_restarts <= 0:
        parser.error("sa-iterations 和 sa-restarts 必须为正整数")
    return Settings(
        data_dir=args.data_dir.expanduser().resolve(),
        output_dir=args.output_dir.expanduser().resolve(),
        sa_iterations=args.sa_iterations,

```

```

        sa_restarts=args.sa_restarts,
        ridge_alpha=args.ridge_alpha,
    )

# ===== 通用函数 =====

def find_attachment(data_dir: Path, number: int) -> Path:
    candidates = sorted(
        data_dir.glob(f"附件{number}*.xlsx"),
        key=lambda path: (len(path.name), path.name),
    )
    if not candidates:
        raise FileNotFoundError(
            f"在 {data_dir} 中没有找到附件{number}*.xlsx。"
            "请使用--data-dir 指定四个附件所在目录。"
        )
    return candidates[0]

def require_columns(frame: pd.DataFrame, required: set[str], source: str) -> None:
    missing = required - set(frame.columns)
    if missing:
        raise ValueError(f"{source}缺少字段: {sorted(missing)}")

def safe_divide(numerator: pd.Series, denominator: pd.Series) -> pd.Series:
    value = numerator.astype(float) / denominator.astype(float).replace(0.0, np.nan)
    return value.replace([np.inf, -np.inf], np.nan)

def weighted_average(values: pd.Series, weights: pd.Series) -> float:
    mask = values.notna() & weights.notna() & (weights > 0)
    if not mask.any():
        return float("nan")
    return float(
        np.average(
            values.loc[mask].astype(float),
            weights=weights.loc[mask].astype(float),
        )
    )

def write_csv(frame: pd.DataFrame, path: Path, index: bool = False) -> None:

```

```

frame.to_csv(path, index=index, encoding="utf-8-sig")

def write_json(data: dict, path: Path) -> None:
    path.write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")

# ===== 数据读取与日级聚合 =====

def load_raw_data(settings: Settings) -> dict[str, pd.DataFrame]:
    paths = {number: find_attachment(settings.data_dir, number) for number in range(1, 5)}
    print("读取附件: ")
    for number, path in paths.items():
        print(f" 附件{number}: {path}")

    items_raw = pd.read_excel(paths[1], dtype={"单品编码": str, "分类编码": str})
    sales_raw = pd.read_excel(paths[2], dtype={"单品编码": str})
    wholesale_raw = pd.read_excel(paths[3], dtype={"单品编码": str})
    loss_excel = pd.ExcelFile(paths[4])
    loss_item_raw = pd.read_excel(paths[4], sheet_name="Sheet1", dtype={"单品编码": str})
    category_sheet = next((name for name in loss_excel.sheet_names if name != "Sheet1"), None)
    if category_sheet is None:
        raise ValueError("附件 4 中没有找到品类平均损耗率工作表")
    loss_category_raw = pd.read_excel(
        paths[4], sheet_name=category_sheet, dtype={"小分类编码": str}
    )

    require_columns(items_raw, {"单品编码", "单品名称", "分类编码", "分类名称"}, "附件 1")
    require_columns(
        sales_raw,
        {
            "销售日期", "单品编码", "销量(千克)", "销售单价(元/千克)",
            "销售类型", "是否打折销售",
        },
        "附件 2",
    )
    require_columns(wholesale_raw, {"日期", "单品编码", "批发价格(元/千克)"}, "附件 3")
    require_columns(loss_item_raw, {"单品编码", "损耗率(%)", "附件 4-Sheet1"})

```

```

require_columns(loss_category_raw, {"小分类编码", "小分类名称"}, f"附件 4-
{category_sheet}")

items = items_raw.rename(columns={
    "单品编码": "item_code",
    "单品名称": "item_name",
    "分类编码": "category_code",
    "分类名称": "category_name",
}).drop_duplicates("item_code")
items["item_code"] = items["item_code"].astype(str).strip()
items["category_code"] = items["category_code"].astype(str).strip()

sales = sales_raw.rename(columns={
    "销售日期": "sale_date",
    "单品编码": "item_code",
    "销量(千克)": "quantity_kg_raw",
    "销售单价(元/千克)": "unit_price_cny_per_kg",
    "销售类型": "sale_type",
    "是否打折销售": "discount_flag_raw",
}).copy()
sales["item_code"] = sales["item_code"].astype(str).strip()
sales["sale_date"] = pd.to_datetime(sales["sale_date"],
errors="coerce").dt.normalize()
sales["quantity_kg_raw"] = pd.to_numeric(
    sales["quantity_kg_raw"], errors="coerce"
).fillna(0.0)
sales["unit_price_cny_per_kg"] = pd.to_numeric(
    sales["unit_price_cny_per_kg"], errors="coerce"
)
sales = sales.loc[sales["sale_date"].between(HIST_START, HIST_END)].copy()

is_return = sales["sale_type"].astype(str).str.contains("退货", na=False)
sales["sale_qty_kg"] = np.where(
    ~is_return & (sales["quantity_kg_raw"] > 0),
    sales["quantity_kg_raw"],
    0.0,
)
sales["return_qty_kg"] = np.where(
    is_return, np.abs(sales["quantity_kg_raw"]), 0.0
)
sales["net_qty_kg"] = sales["sale_qty_kg"] - sales["return_qty_kg"]
sales["positive_sale_value_cny"] = (
    sales["sale_qty_kg"] * sales["unit_price_cny_per_kg"].fillna(0.0)

```

```

)
discount_text = sales["discount_flag_raw"].astype(str).strip().str.lower()
sales["is_discount"] = discount_text.isin({"是", "1", "true", "yes", "y"})

wholesale = wholesale_raw.rename(columns={
    "日期": "date",
    "单品编码": "item_code",
    "批发价格(元/千克)": "wholesale_price_cny_per_kg",
}).copy()
wholesale["item_code"] = wholesale["item_code"].astype(str).strip()
wholesale["date"] = pd.to_datetime(wholesale["date"],
errors="coerce").dt.normalize()
wholesale["wholesale_price_cny_per_kg"] = pd.to_numeric(
    wholesale["wholesale_price_cny_per_kg"], errors="coerce"
)
wholesale = wholesale.dropna(
    subset=["date", "item_code", "wholesale_price_cny_per_kg"]
)
wholesale = wholesale.groupby(
    ["date", "item_code"], as_index=False
)["wholesale_price_cny_per_kg"].mean()

loss_value_column = next(
    (column for column in loss_category_raw.columns if "损耗率" in str(column)),
    None,
)
if loss_value_column is None:
    raise ValueError(f"附件 4-{category_sheet}中没有找到损耗率字段")
loss_category = loss_category_raw.rename(columns={
    "小分类编码": "category_code",
    "小分类名称": "category_name",
    loss_value_column: "loss_rate_pct",
}).copy()
loss_category["category_code"] =
loss_category["category_code"].astype(str).strip()
loss_category["loss_rate_pct"] = pd.to_numeric(
    loss_category["loss_rate_pct"], errors="coerce"
)
loss_category["loss_rate"] = loss_category["loss_rate_pct"] / 100.0

return {
    "items": items.reset_index(drop=True),
    "sales": sales.reset_index(drop=True),

```

```

        "wholesale": wholesale.reset_index(drop=True),
        "loss_category": loss_category.reset_index(drop=True),
    }

def build_daily_item_data(raw: dict[str, pd.DataFrame]) -> pd.DataFrame:
    sales = raw["sales"]
    daily = sales.groupby(["sale_date", "item_code"], as_index=False).agg(
        gross_sale_qty_kg=("sale_qty_kg", "sum"),
        return_qty_kg=("return_qty_kg", "sum"),
        net_qty_kg=("net_qty_kg", "sum"),
        positive_sale_value_cny=("positive_sale_value_cny", "sum"),
        transaction_count=("item_code", "size"),
        discount_transaction_count=("is_discount", "sum"),
    )
    daily["weighted_avg_sale_price_cny_per_kg"] = safe_divide(
        daily["positive_sale_value_cny"], daily["gross_sale_qty_kg"]
    )
    daily = daily.merge(
        raw["items"], on="item_code", how="left", validate="many_to_one"
    )
    daily = daily.merge(
        raw["wholesale"],
        left_on=["sale_date", "item_code"],
        right_on=["date", "item_code"],
        how="left",
        validate="many_to_one",
    ).drop(columns="date")
    daily = daily.sort_values(["item_code", "sale_date"]).reset_index(drop=True)

    # 批发价只在同一单品内部插值，不能借用其他单品的价格。
    daily["wholesale_price_cny_per_kg"] = daily.groupby(
        "item_code", sort=False
    )["wholesale_price_cny_per_kg"].transform(
        lambda series: series.interpolate(limit_direction="both").ffill().bfill()
    )
    return daily

# ===== 价格指数与品类面板 =====

def build_category_panel(
    raw: dict[str, pd.DataFrame],
    daily: pd.DataFrame,

```



```

    output_dir: Path,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    positive = daily.loc[
        (daily["net_qty_kg"] > 0)
        & (daily["weighted_avg_sale_price_cny_per_kg"] > 0)
        & daily["wholesale_price_cny_per_kg"].notna()
    ].copy()
    if positive.empty:
        raise ValueError("没有可用于构造品类价格指数的正销量记录")

    positive["sale_value"] = (
        positive["net_qty_kg"] * positive["weighted_avg_sale_price_cny_per_kg"]
    )
    base_item = positive.groupby(
        ["item_code", "category_code", "category_name"], as_index=False
    ).agg(total_qty=("net_qty_kg", "sum"), total_value=("sale_value", "sum"))
    base_item["item_base_price"] = safe_divide(
        base_item["total_value"], base_item["total_qty"]
    )

    category_base_rows: list[dict] = []
    for (code, name), group in base_item.groupby(
        ["category_code", "category_name"], sort=False
    ):
        category_base_rows.append({
            "category_code": code,
            "category_name": name,
            "category_base_price": weighted_average(
                group["item_base_price"], group["total_qty"]
            ),
        })
    category_base = pd.DataFrame(category_base_rows)
    base_item = base_item.merge(
        category_base, on=["category_code", "category_name"], how="left"
    )
    positive = positive.merge(
        base_item[["item_code", "item_base_price", "category_base_price"]],
        on="item_code",
        how="left",
    )
    positive["weighted_relative_price"] = positive["net_qty_kg"] * (
        positive["weighted_avg_sale_price_cny_per_kg"]
        / positive["item_base_price"]
    )

```

```

)

positive["cost_value"] = (
    positive["net_qty_kg"] * positive["wholesale_price_cny_per_kg"]
)
base_cost = positive.groupby(
    ["item_code", "category_code"], as_index=False
).agg(total_qty=("net_qty_kg", "sum"), total_cost=("cost_value", "sum"))
base_cost["item_base_cost"] = safe_divide(
    base_cost["total_cost"], base_cost["total_qty"]
)
category_cost_rows: list[dict] = []
for code, group in base_cost.groupby("category_code", sort=False):
    category_cost_rows.append({
        "category_code": code,
        "category_base_cost": weighted_average(
            group["item_base_cost"], group["total_qty"]
        ),
    })
category_base_cost = pd.DataFrame(category_cost_rows)
base_cost = base_cost.merge(category_base_cost, on="category_code", how="left")
positive = positive.merge(
    base_cost[["item_code", "item_base_cost", "category_base_cost"]],
    on="item_code",
    how="left",
)
positive["weighted_relative_cost"] = positive["net_qty_kg"] * (
    positive["wholesale_price_cny_per_kg"] / positive["item_base_cost"]
)

daily_category = positive.groupby(
    ["sale_date", "category_code", "category_name"], as_index=False
).agg(
    quantity_kg=("net_qty_kg", "sum"),
    weighted_price_sum=("weighted_relative_price", "sum"),
    weighted_cost_sum=("weighted_relative_cost", "sum"),
    category_base_price=("category_base_price", "first"),
    category_base_cost=("category_base_cost", "first"),
)
daily_category["price_index"] = daily_category["category_base_price"] *
safe_divide(
    daily_category["weighted_price_sum"], daily_category["quantity_kg"]
)

```

```

    daily_category["unit_cost"] = daily_category["category_base_cost"] *
safe_divide(
    daily_category["weighted_cost_sum"], daily_category["quantity_kg"]
)

panel_parts: list[pd.DataFrame] = []
for (code, name), group in daily_category.groupby(
    ["category_code", "category_name"], sort=False
):
    panel = pd.DataFrame({"sale_date": FULL_CALENDAR})
    panel = panel.merge(
        group.drop(columns=["category_code", "category_name"]),
        on="sale_date",
        how="left",
    )
    panel["category_code"] = code
    panel["category_name"] = name
    panel["quantity_kg"] = panel["quantity_kg"].fillna(0.0).clip(lower=0.0)
    # 无交易日销量补 0; 价格和成本不能补 0, 采用时间插值。
    panel["price_index"] = (
        panel["price_index"].interpolate(limit_direction="both").ffill().bfill()
    )
    panel["unit_cost"] = (
        panel["unit_cost"].interpolate(limit_direction="both").ffill().bfill()
    )
    panel_parts.append(panel)
category_panel = pd.concat(panel_parts, ignore_index=True)

sales_with_category = raw["sales"].merge(
    raw["items"][["item_code", "category_code", "category_name"]],
    on="item_code",
    how="left",
)
positive_sales = sales_with_category.loc[
    sales_with_category["sale_qty_kg"] > 0
].copy()
discount_rows: list[dict] = []
for (code, name), group in positive_sales.groupby(
    ["category_code", "category_name"], sort=False
):
    total = float(group["sale_qty_kg"].sum())
    discounted = float(group.loc[group["is_discount"], "sale_qty_kg"].sum())
    discount_rows.append({

```

```

        "category_code": code,
        "category_name": name,
        "discount_volume_share": discounted / total if total > 0 else 0.0,
    })
discount_share = pd.DataFrame(discount_rows)

write_csv(category_panel, output_dir / "q2_category_daily_panel.csv")
write_csv(base_item, output_dir / "q2_item_base_price.csv")
write_csv(category_base, output_dir / "q2_category_base_price.csv")
write_csv(discount_share, output_dir / "q2_category_discount_share.csv")
return category_panel, discount_share

# ===== 加性分解与 FWL/OLS =====

def holiday_flag(dates: pd.DatetimeIndex) -> np.ndarray:
    flag = np.zeros(len(dates), dtype=bool)
    for start, end in HOLIDAY_INTERVALS:
        flag |= (dates >= pd.Timestamp(start)) & (dates <= pd.Timestamp(end))
    return flag.astype(float)

def time_design(dates: pd.DatetimeIndex) -> tuple[np.ndarray, dict[str, slice]]:
    """分段线性趋势 + 周 Fourier 项 + 年 Fourier 项 + 节假日脉冲。"""
    dates = pd.DatetimeIndex(dates)
    total_days = float((HIST_END - HIST_START).days)
    normalized_time = (dates - HIST_START).days.to_numpy(dtype=float) / total_days
    day_number = (dates - HIST_START).days.to_numpy(dtype=float)

    parts: list[np.ndarray] = []
    groups: dict[str, slice] = {}
    trend = np.column_stack(
        [np.ones(len(dates)), normalized_time]
        + [
            np.maximum(normalized_time - change_point, 0.0)
            for change_point in np.linspace(0.1, 0.9, 8)
        ]
    )
    groups["trend"] = slice(0, trend.shape[1])
    parts.append(trend)
    start = trend.shape[1]

    weekly = np.column_stack([
        function(2.0 * np.pi * harmonic * day_number / 7.0)

```

```

        for harmonic in range(1, 4)
        for function in (np.sin, np.cos)
    ])
    groups["weekly"] = slice(start, start + weekly.shape[1])
    parts.append(weekly)
    start += weekly.shape[1]

    yearly = np.column_stack([
        function(2.0 * np.pi * harmonic * day_number / 365.25)
        for harmonic in range(1, 11)
        for function in (np.sin, np.cos)
    ])
    groups["yearly"] = slice(start, start + yearly.shape[1])
    parts.append(yearly)
    start += yearly.shape[1]

    holiday = holiday_flag(dates).reshape(-1, 1)
    groups["holiday"] = slice(start, start + 1)
    parts.append(holiday)
    return np.column_stack(parts), groups

def smape(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    denominator = np.abs(y_true) + np.abs(y_pred)
    error = np.where(
        denominator > 1e-9,
        2.0 * np.abs(y_true - y_pred) / denominator,
        0.0,
    )
    return float(np.mean(error) * 100.0)

def fit_partial_linear(
    dates: pd.DatetimeIndex,
    quantity: np.ndarray,
    price: np.ndarray,
    ridge_alpha: float,
    train_mask: np.ndarray | None = None,
) -> dict:
    dates = pd.DatetimeIndex(dates)
    design, groups = time_design(dates)
    if train_mask is None:
        train_mask = np.ones(len(quantity), dtype=bool)
    train_mask = np.asarray(train_mask, dtype=bool)

```

```

price_p10, price_p90 = np.quantile(price[train_mask], [0.10, 0.90])
local_mask = train_mask & (price >= price_p10) & (price <= price_p90)
reference_price = float(np.median(price[train_mask]))
centered_price = price - reference_price

quantity_time_model = Ridge(
    alpha=ridge_alpha, fit_intercept=False
).fit(design[train_mask], quantity[train_mask])
price_time_model = Ridge(
    alpha=ridge_alpha, fit_intercept=False
).fit(design[train_mask], centered_price[train_mask])

quantity_residual = quantity - quantity_time_model.predict(design)
price_residual = centered_price - price_time_model.predict(design)
denominator = float(np.dot(price_residual[local_mask],
price_residual[local_mask]))
unrestricted_beta = (
    float(
        np.dot(
            price_residual[local_mask], quantity_residual[local_mask]
        ) / denominator
    )
    if denominator > 1e-12
    else 0.0
)
# 需求价格效应设为非正；本题六品类无约束估计本身均为负。
beta = min(unrestricted_beta, -1e-6)

time_target = quantity - beta * centered_price
baseline_model = Ridge(
    alpha=ridge_alpha, fit_intercept=False
).fit(design[train_mask], time_target[train_mask])
prediction = baseline_model.predict(design) + beta * centered_price

local_total = float(
    np.sum(
        (quantity_residual[local_mask] - quantity_residual[local_mask].mean()) **
2
    )
)
local_residual_sum = float(
    np.sum(

```

```

        (quantity_residual[local_mask] - beta * price_residual[local_mask]) ** 2
    )
)
local_r2 = (
    1.0 - local_residual_sum / local_total
    if local_total > 1e-12
    else float("nan")
)
return {
    "beta": beta,
    "unrestricted_beta": unrestricted_beta,
    "reference_price": reference_price,
    "price_p10": float(price_p10),
    "price_p90": float(price_p90),
    "local_r2": local_r2,
    "baseline_model": baseline_model,
    "prediction": prediction,
    "design": design,
    "groups": groups,
}

def validate_category_model(
    dates: pd.DatetimeIndex,
    quantity: np.ndarray,
    price: np.ndarray,
    ridge_alpha: float,
) -> dict:
    # 最后 84 天为测试集，其余日期为训练集。
    cutoff = dates.max() - pd.Timedelta(days=83)
    train_mask = dates < cutoff
    fit = fit_partial_linear(
        dates, quantity, price, ridge_alpha=ridge_alpha, train_mask=train_mask
    )
    test_mask = ~train_mask
    prediction = np.clip(fit["prediction"][test_mask], 0.0, None)
    actual = quantity[test_mask]

    seasonal_naive_values: list[float] = []
    for date in dates[test_mask]:
        previous_index = np.where(dates == date - pd.Timedelta(days=7))[0]
        if len(previous_index):
            seasonal_naive_values.append(float(quantity[previous_index[0]]))

```

```

        else:
            seasonal_naive_values.append(float(np.mean(quantity[train_mask])))
seasonal_naive = np.asarray(seasonal_naive_values, dtype=float)

return {
    "mae_model": float(mean_absolute_error(actual, prediction)),
    "rmse_model": float(math.sqrt(mean_squared_error(actual, prediction))),
    "smape_model_pct": smape(actual, prediction),
    "mae_seasonal_naive": float(mean_absolute_error(actual, seasonal_naive)),
    "rmse_seasonal_naive": float(
        math.sqrt(mean_squared_error(actual, seasonal_naive))
    ),
    "smape_seasonal_naive_pct": smape(actual, seasonal_naive),
}

def build_component_table(
    category_code: str,
    category_name: str,
    dates: pd.DatetimeIndex,
    price: np.ndarray,
    fit: dict,
) -> pd.DataFrame:
    coefficients = fit["baseline_model"].coef_
    design = fit["design"]
    groups = fit["groups"]
    return pd.DataFrame({
        "date": dates,
        "category_code": category_code,
        "category_name": category_name,
        "trend": design[:, groups["trend"]] @ coefficients[groups["trend"]],
        "weekly": design[:, groups["weekly"]] @ coefficients[groups["weekly"]],
        "yearly": design[:, groups["yearly"]] @ coefficients[groups["yearly"]],
        "holiday": design[:, groups["holiday"]] @ coefficients[groups["holiday"]],
        "price_effect": fit["beta"] * (price - fit["reference_price"]),
        "fitted_demand": fit["prediction"],
    })

# ===== 利润函数与模拟退火 =====

def category_profit(
    state: np.ndarray,
    baseline_demand: float,

```



```

    price_beta: float,
    reference_price: float,
    loss_rate: float,
    unit_cost: float,
    discount_share: float,
) -> tuple[float, float, float, float, float]:
    price, replenishment, discount_ratio = state
    regular_demand = max(
        0.0,
        baseline_demand + price_beta * (price - reference_price),
    )
    sellable_supply = max(0.0, (1.0 - loss_rate) * replenishment)
    regular_sales = min(regular_demand, sellable_supply)
    leftover = max(0.0, sellable_supply - regular_sales)
    clearance_demand = max(
        0.0,
        discount_share * baseline_demand
        + abs(price_beta) * (1.0 - discount_ratio) * price,
    )
    discount_sales = min(leftover, clearance_demand)
    profit = (
        price * regular_sales
        + discount_ratio * price * discount_sales
        - unit_cost * replenishment
    )
    return profit, regular_sales, discount_sales, regular_demand, sellable_supply

def simulated_annealing(
    bounds: list[tuple[float, float]],
    objective: Callable[[np.ndarray], float],
    seed: int,
    iterations: int,
    restarts: int,
    initial_temperature: float = 20.0,
    cooling: float = 0.997,
) -> tuple[np.ndarray, float, list[float]]:
    rng = np.random.default_rng(seed)
    lower = np.asarray([bound[0] for bound in bounds], dtype=float)
    upper = np.asarray([bound[1] for bound in bounds], dtype=float)
    span = upper - lower
    global_best_state: np.ndarray | None = None
    global_best_value = -np.inf

```

```

saved_curve: list[float] = []

for restart in range(restarts):
    state = lower + rng.random(len(bounds)) * span
    value = objective(state)
    best_state = state.copy()
    best_value = value
    temperature = initial_temperature
    curve: list[float] = []

    for iteration in range(iterations):
        search_scale = 0.12 * max(temperature / initial_temperature, 0.03)
        candidate = np.clip(
            state + rng.normal(0.0, search_scale, len(bounds)) * span,
            lower,
            upper,
        )
        candidate_value = objective(candidate)
        exponent = np.clip(
            (candidate_value - value) / max(temperature, 1e-9),
            -700.0,
            0.0,
        )
        if candidate_value >= value or rng.random() < math.exp(exponent):
            state, value = candidate, candidate_value
        if value < best_value:
            best_state, best_value = state.copy(), value
        if restart == 0 and (iteration % 50 == 0 or iteration == iterations - 1):
            curve.append(best_value)
        temperature *= cooling

    if best_value < global_best_value:
        global_best_state = best_state.copy()
        global_best_value = best_value
    if restart == 0:
        saved_curve = curve

if global_best_state is None:
    raise RuntimeError("模拟退火没有产生可用解")
return global_best_state, float(global_best_value), saved_curve

```

```

# ===== 问题二主计算 =====

```

```

def run_problem2(
    raw: dict[str, pd.DataFrame],
    panel: pd.DataFrame,
    discount_share: pd.DataFrame,
    settings: Settings,
) -> dict:
    coefficient_rows: list[dict] = []
    validation_rows: list[dict] = []
    decision_rows: list[dict] = []
    component_parts: list[pd.DataFrame] = []
    convergence_rows: list[dict] = []
    future_design, _ = time_design(FUTURE_DATES)

    grouped = panel.groupby(["category_code", "category_name"], sort=False)
    for category_index, ((code, name), group) in enumerate(grouped):
        print(f"拟合品类: {name}")
        group = group.sort_values("sale_date")
        dates = pd.DatetimeIndex(group["sale_date"])
        quantity = group["quantity_kg"].to_numpy(dtype=float, copy=True)
        price = group["price_index"].to_numpy(dtype=float, copy=True)

        fit = fit_partial_linear(
            dates, quantity, price, ridge_alpha=settings.ridge_alpha
        )
        validation = validate_category_model(
            dates, quantity, price, ridge_alpha=settings.ridge_alpha
        )
        validation.update({"category_code": code, "category_name": name})
        validation_rows.append(validation)
        component_parts.append(
            build_component_table(code, name, dates, price, fit)
        )

    discount_match = discount_share.loc[
        discount_share["category_code"] == code, "discount_volume_share"
    ]
    loss_match = raw["loss_category"].loc[
        raw["loss_category"]["category_code"] == code, "loss_rate"
    ]
    discount_ratio_history = (
        float(discount_match.iloc[0]) if len(discount_match) else 0.0
    )

```

```

loss_rate = float(loss_match.iloc[0]) if len(loss_match) else 0.0

recent_mask = group["sale_date"].between("2023-06-24", "2023-06-30")
recent_unit_cost = float(group.loc[recent_mask, "unit_cost"].mean())
recent_quantity = float(group.loc[recent_mask, "quantity_kg"].mean())
baseline_future = np.clip(
    fit["baseline_model"].predict(future_design), 0.0, None
)

coefficient_rows.append({
    "category_code": code,
    "category_name": name,
    "price_beta_kg_per_yuan": fit["beta"],
    "unrestricted_beta": fit["unrestricted_beta"],
    "local_r2": fit["local_r2"],
    "price_p10": fit["price_p10"],
    "reference_price": fit["reference_price"],
    "price_p90": fit["price_p90"],
    "discount_volume_share": discount_ratio_history,
    "loss_rate": loss_rate,
    "recent_7d_unit_cost": recent_unit_cost,
})

for day_index, (date, baseline) in enumerate(
    zip(FUTURE_DATES, baseline_future)
):
    replenishment_upper = max(
        5.0,
        1.60
        * max(float(baseline), recent_quantity, 1.0)
        / max(1.0 - loss_rate, 1e-6),
    )
    bounds = [
        (fit["price_p10"], fit["price_p90"]),
        (0.0, replenishment_upper),
        (0.60, 0.95),
    ]

    def objective(state: np.ndarray) -> float:
        return category_profit(
            state,
            float(baseline),
            fit["beta"],

```

```

        fit["reference_price"],
        loss_rate,
        recent_unit_cost,
        discount_ratio_history,
    )[0]

best_state, _, convergence_curve = simulated_annealing(
    bounds=bounds,
    objective=objective,
    seed=SEED + category_index * 100 + day_index,
    iterations=settings.sa_iterations,
    restarts=settings.sa_restarts,
)
profit, regular_sales, discount_sales, demand, supply = category_profit(
    best_state,
    float(baseline),
    fit["beta"],
    fit["reference_price"],
    loss_rate,
    recent_unit_cost,
    discount_ratio_history,
)
decision_rows.append({
    "date": date.date().isoformat(),
    "category_code": code,
    "category_name": name,
    "baseline_demand_at_reference_kg": float(baseline),
    "optimal_price_index": float(best_state[0]),
    "optimal_replenishment_kg": float(best_state[1]),
    "optimal_discount_ratio": float(best_state[2]),
    "regular_sales_kg": regular_sales,
    "discount_sales_kg": discount_sales,
    "sellable_supply_kg": supply,
    "expected_demand_kg": demand,
    "expected_profit_cny": profit,
})
# 每个品类只保存 7 月 1 日第一次重启的收敛曲线，避免结果文件过大。
if day_index == 0:
    convergence_rows.extend({
        "category_code": code,
        "category_name": name,
        "record_index": index + 1,
        "iteration": min(index * 50 + 1, settings.sa_iterations),
    })

```

```

        "best_profit_cny": value,
    } for index, value in enumerate(convergence_curve))

coefficients = pd.DataFrame(coefficient_rows)
validation = pd.DataFrame(validation_rows)
decisions = pd.DataFrame(decision_rows)
components = pd.concat(component_parts, ignore_index=True)
convergence = pd.DataFrame(convergence_rows)

category_summary = decisions.groupby(
    ["category_code", "category_name"], as_index=False
).agg(
    average_price_index=("optimal_price_index", "mean"),
    total_replenishment_kg=("optimal_replenishment_kg", "sum"),
    average_discount_ratio=("optimal_discount_ratio", "mean"),
    total_regular_sales_kg=("regular_sales_kg", "sum"),
    total_discount_sales_kg=("discount_sales_kg", "sum"),
    total_expected_profit_cny=("expected_profit_cny", "sum"),
)
daily_summary = decisions.groupby("date", as_index=False).agg(
    total_replenishment_kg=("optimal_replenishment_kg", "sum"),
    total_regular_sales_kg=("regular_sales_kg", "sum"),
    total_discount_sales_kg=("discount_sales_kg", "sum"),
    total_expected_profit_cny=("expected_profit_cny", "sum"),
)

write_csv(coefficients, settings.output_dir / "q2_category_price_effects.csv")
write_csv(validation, settings.output_dir / "q2_validation.csv")
write_csv(decisions, settings.output_dir / "q2_2023-07-01_to_07_decisions.csv")
write_csv(components, settings.output_dir / "q2_additive_components.csv")
write_csv(convergence, settings.output_dir / "q2_sa_convergence.csv")
write_csv(category_summary, settings.output_dir / "q2_category_7d_summary.csv")
write_csv(daily_summary, settings.output_dir / "q2_daily_7d_summary.csv")

summary = {
    "total_7d_replenishment_kg": float(
        decisions["optimal_replenishment_kg"].sum()
    ),
    "total_7d_profit_cny": float(decisions["expected_profit_cny"].sum()),
    "mean_discount_ratio": float(decisions["optimal_discount_ratio"].mean()),
    "all_unrestricted_price_betas_negative": bool(
        (coefficients["unrestricted_beta"] < 0).all()
    ),
}

```

```

        "sa_iterations_per_run": settings.sa_iterations,
        "sa_restarts": settings.sa_restarts,
        "ridge_alpha": settings.ridge_alpha,
    }
    write_json(summary, settings.output_dir / "q2_summary.json")
    return summary

def main() -> None:
    settings = parse_args()
    settings.output_dir.mkdir(parents=True, exist_ok=True)
    raw = load_raw_data(settings)
    daily = build_daily_item_data(raw)
    panel, discount_share = build_category_panel(
        raw, daily, settings.output_dir
    )
    summary = run_problem2(raw, panel, discount_share, settings)

    print("\n 问题二计算完成: ")
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    print(f"结果目录: {settings.output_dir}")

if __name__ == "__main__":
    main()

```

问题三

```

from __future__ import annotations

import argparse
import json
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import differential_evolution

try:
    import matplotlib.font_manager as font_manager
    import matplotlib.pyplot as plt
except ImportError:
    font_manager = None

```

```

plt = None

SEED = 20230818
CANDIDATE_START = pd.Timestamp("2023-06-24")
CANDIDATE_END = pd.Timestamp("2023-06-30")
RECENT_START = pd.Timestamp("2023-06-03")
DECISION_DATE = pd.Timestamp("2023-07-01")
MIN_DISPLAY_KG = 2.5
MIN_ITEM_COUNT = 27
MAX_ITEM_COUNT = 33
DEFAULT_RESULT_DIR = Path(r"D:\code\problem1_result")
FIGURE_DPI = 300

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="运行 2023 C 题问题三单品定价与补货优化")
    parser.add_argument("--data-dir", type=Path, default=Path("."), help="附件 1--4 所在目录")
    parser.add_argument(
        "--q2-results-dir", type=Path, default=DEFAULT_RESULT_DIR,
        help=r"问题二计算结果所在目录，默认 D:\code\problem1_result",
    )
    parser.add_argument(
        "--output-dir", type=Path, default=DEFAULT_RESULT_DIR,
        help=r"问题三表格和图片输出目录，默认 D:\code\problem1_result",
    )
    parser.add_argument("--de-maxiter", type=int, default=140, help="差分进化最大代数")
    parser.add_argument("--de-popsiz", type=int, default=6, help="差分进化种群倍数")
    args = parser.parse_args()
    args.data_dir = args.data_dir.expanduser().resolve()
    args.q2_results_dir = args.q2_results_dir.expanduser().resolve()
    args.output_dir = args.output_dir.expanduser().resolve()
    return args

def find_attachment(data_dir: Path, number: int) -> Path:
    candidates = sorted(data_dir.glob(f"附件{number}*.xlsx"), key=lambda p:
        (len(p.name), p.name))
    if not candidates:
        raise FileNotFoundError(f"在 {data_dir} 中没有找到附件{number}*.xlsx")
    return candidates[0]

```



```

def require_file(directory: Path, filename: str) -> Path:
    path = directory / filename
    if not path.exists():
        raise FileNotFoundError(f"缺少 {path}, 请先运行问题二建模代码。")
    return path

def require_columns(frame: pd.DataFrame, required: set[str], source: str) -> None:
    missing = required - set(frame.columns)
    if missing:
        raise ValueError(f"{source} 缺少字段: {sorted(missing)}")

def safe_divide(numerator: pd.Series, denominator: pd.Series) -> pd.Series:
    result = numerator.astype(float) / denominator.astype(float).replace(0.0,
np.nan)
    return result.replace([np.inf, -np.inf], np.nan)

def write_csv(frame: pd.DataFrame, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(path, index=False, encoding="utf-8-sig")

def normalize_columns(frame: pd.DataFrame) -> pd.DataFrame:
    """去除 Excel/CSV 表头中常见的空格、换行和 BOM。"""
    result = frame.copy()
    result.columns = [str(c).replace("\ufeff", "").replace("\n", "").strip() for c
in result.columns]
    return result

def configure_chinese_font() -> None:
    if plt is None or font_manager is None:
        return
    preferred = ["Microsoft YaHei", "SimHei", "SimSun", "Noto Sans CJK SC", "Source
Han Sans SC"]
    installed = {f.name for f in font_manager.fontManager.ttflist}
    selected = next((name for name in preferred if name in installed), None)
    if selected:
        plt.rcParams["font.sans-serif"] = [selected, "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False

```

```

def save_figure(fig, output_dir: Path, stem: str) -> None:
    """同时保存 300 dpi PNG 和矢量 PDF。"""
    output_dir.mkdir(parents=True, exist_ok=True)
    fig.tight_layout()
    fig.savefig(output_dir / f"{stem}.png", dpi=FIGURE_DPI, bbox_inches="tight",
facecolor="white")
    fig.savefig(output_dir / f"{stem}.pdf", bbox_inches="tight", facecolor="white")
    plt.close(fig)

def export_excel_workbook(
    output_dir: Path,
    candidates: pd.DataFrame,
    all_candidates: pd.DataFrame,
    selected_items: pd.DataFrame,
    convergence: pd.DataFrame,
    summary: dict,
) -> None:
    """把主要结果合并到一个 Excel 工作簿；缺少 openpyxl 时保留 CSV，不中断。"""
    target = output_dir / "问题三结果汇总.xlsx"
    try:
        with pd.ExcelWriter(target, engine="openpyxl") as writer:
            candidates.to_excel(writer, sheet_name="候选参数", index=False)
            all_candidates.to_excel(writer, sheet_name="全部候选结果", index=False)
            selected_items.to_excel(writer, sheet_name="入选单品", index=False)
            convergence.to_excel(writer, sheet_name="收敛过程", index=False)
            pd.json_normalize(summary, sep=".").T.reset_index().rename(
                columns={"index": "指标", 0: "数值"}
            ).to_excel(writer, sheet_name="结果摘要", index=False)
    except ImportError:
        print("提示：未安装 openpyxl，已跳过 Excel 汇总文件；CSV 结果仍已正常保存。")

def plot_problem3_results(
    output_dir: Path,
    all_candidates: pd.DataFrame,
    selected_items: pd.DataFrame,
    convergence: pd.DataFrame,
) -> None:
    """生成问题三核心结果图。"""
    if plt is None:

```

```

        print("提示: 未安装 matplotlib, 已跳过图片生成。可执行 python -m pip install
matplotlib")
        return

    configure_chinese_font()

    if not convergence.empty:
        fig, ax = plt.subplots(figsize=(7.4, 4.5))
        ax.plot(convergence["iteration"], convergence["best_profit_cny"],
linewidth=2)
        ax.set_title("问题三差分进化最优利润收敛曲线")
        ax.set_xlabel("迭代代数")
        ax.set_ylabel("当前最优期望利润 (元)")
        ax.grid(alpha=0.25)
        save_figure(fig, output_dir, "图_问题三_差分进化收敛曲线")

    if not selected_items.empty:
        top = selected_items.nlargest(15,
"optimal_replenishment_kg").sort_values("optimal_replenishment_kg")
        labels = top["item_name"].astype(str).str.slice(0, 14)
        fig, ax = plt.subplots(figsize=(9, 6))
        ax.barh(labels, top["optimal_replenishment_kg"])
        ax.set_title("问题三入选单品补货量前 15 名")
        ax.set_xlabel("最优补货量 (kg)")
        ax.set_ylabel("单品")
        ax.grid(axis="x", alpha=0.25)
        save_figure(fig, output_dir, "图_问题三_入选单品补货量前 15 名")

    if not all_candidates.empty and "selected" in all_candidates.columns:
        counts = all_candidates.groupby(["category_name",
"selected"]).size().unstack(fill_value=0)
        for col in (0, 1):
            if col not in counts.columns:
                counts[col] = 0
        counts = counts[[0, 1]].rename(columns={0: "未入选", 1: "入选"})
        fig, ax = plt.subplots(figsize=(8, 4.8))
        counts.plot(kind="bar", stacked=True, ax=ax)
        ax.set_title("问题三候选单品的品类构成与选择结果")
        ax.set_xlabel("品类")
        ax.set_ylabel("单品数")
        ax.tick_params(axis="x", rotation=20)
        ax.legend(frameon=False)
        save_figure(fig, output_dir, "图_问题三_候选单品选择结构")

```

```

def load_inputs(data_dir: Path, q2_results_dir: Path) -> dict[str, pd.DataFrame]:
    attachment1 = find_attachment(data_dir, 1)
    attachment2 = find_attachment(data_dir, 2)
    attachment3 = find_attachment(data_dir, 3)
    attachment4 = find_attachment(data_dir, 4)

    items_raw = pd.read_excel(attachment1, dtype={"单品编码": str, "分类编码": str})
    sales_raw = pd.read_excel(attachment2, dtype={"单品编码": str})
    wholesale_raw = pd.read_excel(attachment3, dtype={"单品编码": str})
    try:
        loss_raw = pd.read_excel(attachment4, sheet_name="Sheet1", dtype={"单品编码":
str})
    except ValueError:
        # 某些附件 4 工作表名称不是 Sheet1, 自动读取第一个工作表。
        loss_raw = pd.read_excel(attachment4, sheet_name=0, dtype={"单品编码": str})

    items_raw = normalize_columns(items_raw)
    sales_raw = normalize_columns(sales_raw)
    wholesale_raw = normalize_columns(wholesale_raw)
    loss_raw = normalize_columns(loss_raw)

    require_columns(items_raw, {"单品编码", "单品名称", "分类编码", "分类名称"}, "附件
1")
    require_columns(
        sales_raw,
        {"销售日期", "单品编码", "销量(千克)", "销售单价(元/千克)", "销售类型"},
        "附件 2",
    )
    require_columns(wholesale_raw, {"日期", "单品编码", "批发价格(元/千克)"}, "附件 3")
    require_columns(loss_raw, {"单品编码", "损耗率(%)"}, "附件 4-Sheet1")

    items = items_raw.rename(columns={
        "单品编码": "item_code", "单品名称": "item_name",
        "分类编码": "category_code", "分类名称": "category_name",
    }).drop_duplicates("item_code")
    items["item_code"] = items["item_code"].astype(str).str.strip()
    items["category_code"] = items["category_code"].astype(str).str.strip()

    sales = sales_raw.rename(columns={
        "销售日期": "sale_date", "单品编码": "item_code", "销量(千克)":
"quantity_kg_raw",

```

```

        "销售单价(元/千克)": "unit_price_cny_per_kg", "销售类型": "sale_type",
    }).copy()
    sales["item_code"] = sales["item_code"].astype(str).str.strip()
    sales["sale_date"] = pd.to_datetime(sales["sale_date"],
errors="coerce").dt.normalize()
    sales["quantity_kg_raw"] = pd.to_numeric(sales["quantity_kg_raw"],
errors="coerce").fillna(0.0)
    sales["unit_price_cny_per_kg"] = pd.to_numeric(sales["unit_price_cny_per_kg"],
errors="coerce")
    is_return = sales["sale_type"].astype(str).str.contains("退货", na=False)
    sales["sale_qty_kg"] = np.where(
        ~is_return & (sales["quantity_kg_raw"] > 0), sales["quantity_kg_raw"], 0.0
    )
    sales["return_qty_kg"] = np.where(is_return, np.abs(sales["quantity_kg_raw"]),
0.0)
    sales["net_qty_kg"] = sales["sale_qty_kg"] - sales["return_qty_kg"]
    sales["positive_sale_value_cny"] = sales["sale_qty_kg"] *
sales["unit_price_cny_per_kg"].fillna(0.0)

    daily = sales.groupby(["sale_date", "item_code"], as_index=False).agg(
        gross_sale_qty_kg=("sale_qty_kg", "sum"),
        net_qty_kg=("net_qty_kg", "sum"),
        positive_sale_value_cny=("positive_sale_value_cny", "sum"),
    )
    daily["weighted_avg_sale_price_cny_per_kg"] = safe_divide(
        daily["positive_sale_value_cny"], daily["gross_sale_qty_kg"]
    )
    daily = daily.merge(items, on="item_code", how="left", validate="many_to_one")

    wholesale = wholesale_raw.rename(columns={
        "日期": "date", "单品编码": "item_code", "批发价格(元/千克)":
"wholesale_price_cny_per_kg",
    }).copy()
    wholesale["item_code"] = wholesale["item_code"].astype(str).str.strip()
    wholesale["date"] = pd.to_datetime(wholesale["date"],
errors="coerce").dt.normalize()
    wholesale["wholesale_price_cny_per_kg"] = pd.to_numeric(
        wholesale["wholesale_price_cny_per_kg"], errors="coerce"
    )
    wholesale = wholesale.dropna(subset=["date", "item_code",
"wholesale_price_cny_per_kg"])
    wholesale = wholesale.groupby(["date", "item_code"],
as_index=False)["wholesale_price_cny_per_kg"].mean()

```

```

    loss = loss_raw.rename(columns={"单品编码": "item_code", "损耗率(%)":
"loss_rate_pct"}).copy()
    loss["item_code"] = loss["item_code"].astype(str).str.strip()
    loss["loss_rate_pct"] = pd.to_numeric(loss["loss_rate_pct"], errors="coerce")
    loss["loss_rate"] = (loss["loss_rate_pct"] / 100.0).clip(lower=0.0, upper=0.95)

    q2_effects = pd.read_csv(
        require_file(q2_results_dir, "q2_category_price_effects.csv"),
        dtype={"category_code": str},
    )
    q2_decisions = pd.read_csv(
        require_file(q2_results_dir, "q2_2023-07-01_to_07_decisions.csv"),
        dtype={"category_code": str},
    )
    q2_effects = normalize_columns(q2_effects)
    q2_decisions = normalize_columns(q2_decisions)
    require_columns(q2_effects, {"category_code", "price_beta_kg_per_yuan"}, "问题二
价格效应结果")
    require_columns(
        q2_decisions,
        {"category_code", "date", "baseline_demand_at_reference_kg"},
        "问题二 7 日决策结果",
    )
    q2_effects["category_code"] =
q2_effects["category_code"].astype(str).str.strip()
    q2_decisions["category_code"] =
q2_decisions["category_code"].astype(str).str.strip()
    q2_decisions["date"] = pd.to_datetime(q2_decisions["date"],
errors="coerce").dt.normalize()

    return {
        "items": items, "daily": daily, "wholesale": wholesale, "loss": loss,
        "q2_effects": q2_effects, "q2_decisions": q2_decisions,
    }

def build_candidate_parameters(data: dict[str, pd.DataFrame]) -> pd.DataFrame:
    wholesale_week = data["wholesale"].loc[
        data["wholesale"]["date"].between(CANDIDATE_START, CANDIDATE_END)
    ].copy()
    costs = wholesale_week.groupby("item_code", as_index=False).agg(
        observed_wholesale_days=("date", "nunique"),

```

```

        procurement_cost=("wholesale_price_cny_per_kg", "mean"),
    )
    candidates = (
        costs.merge(data["items"], on="item_code", how="left")
        .merge(data["loss"][["item_code", "loss_rate"]], on="item_code", how="left")
    )

    daily = data["daily"]
    recent = daily.loc[
        daily["sale_date"].between(RECENT_START, CANDIDATE_END) &
        (daily["net_qty_kg"] > 0)
    ].copy()
    recent["sale_value"] = recent["net_qty_kg"] *
recent["weighted_avg_sale_price_cny_per_kg"]
    recent_summary = recent.groupby("item_code", as_index=False).agg(
        recent_qty=("net_qty_kg", "sum"), recent_value=("sale_value", "sum")
    )
    recent_summary["recent_price"] = safe_divide(
        recent_summary["recent_value"], recent_summary["recent_qty"]
    )

    full = daily.loc[daily["net_qty_kg"] > 0].copy()
    full["sale_value"] = full["net_qty_kg"] *
full["weighted_avg_sale_price_cny_per_kg"]
    full_summary = full.groupby("item_code", as_index=False).agg(
        full_qty=("net_qty_kg", "sum"), full_value=("sale_value", "sum")
    )
    full_summary["full_price"] = safe_divide(full_summary["full_value"],
full_summary["full_qty"])

    candidates = candidates.merge(
        recent_summary[["item_code", "recent_qty", "recent_price"]], on="item_code",
how="left"
    )
    candidates = candidates.merge(
        full_summary[["item_code", "full_qty", "full_price"]], on="item_code",
how="left"
    )
    candidates["recent_qty"] = candidates["recent_qty"].fillna(0.0)
    candidates["reference_price"] =
candidates["recent_price"].fillna(candidates["full_price"])
    candidates["reference_price"] = candidates["reference_price"].fillna(
        1.35 * candidates["procurement_cost"]
    )

```

```

)
candidates["loss_rate"] = candidates["loss_rate"].fillna(0.0)

q2_july1 = data["q2_decisions"].loc[data["q2_decisions"]["date"] ==
DECISION_DATE].copy()
if q2_july1["category_code"].nunique() != 6:
    raise ValueError("问题二结果中没有完整的 2023-07-01 六品类预测结果")
category_baseline =
q2_july1.set_index("category_code")["baseline_demand_at_reference_kg"].to_dict()
category_beta =
data["q2_effects"].set_index("category_code")["price_beta_kg_per_yuan"].to_dict()

rows: list[dict] = []
for category_code, group in candidates.groupby("category_code", sort=False):
    if category_code not in category_baseline or category_code not in
category_beta:
        raise ValueError(f"问题二结果缺少品类 {category_code} 的需求或价格系数")
    history = full.loc[full["category_code"] ==
category_code].groupby("item_code")["net_qty_kg"].sum()
    weights = np.asarray([
        row["recent_qty"] if row["recent_qty"] > 0
        else 0.05 * float(history.get(row["item_code"], 0.0)) + 0.01
        for _, row in group.iterrows()
    ], dtype=float)
    weights /= weights.sum()
    for (_, row), share in zip(group.iterrows(), weights):
        record = row.to_dict()
        record["demand_share"] = float(share)
        record["baseline_item_demand_kg"] =
float(category_baseline[category_code]) * float(share)
        record["item_price_beta"] = float(category_beta[category_code]) *
float(share)
        rows.append(record)

if not rows:
    raise ValueError("没有构造出候选单品。请检查附件 3 在 2023-06-24 至 2023-06-30 是
否存在批发价记录。")
result = pd.DataFrame(rows).sort_values(["category_name",
"item_code"]).reset_index(drop=True)
if result["category_name"].isna().any():
    raise ValueError("候选单品中存在无法匹配品类的单品编码，请检查附件 1 与附件 3 的单品
编码。")
if len(result) < MIN_ITEM_COUNT:

```



```

        raise ValueError(f"候选单品只有 {len(result)} 个，少于最少上架数
{MIN_ITEM_COUNT}，无法满足约束。")
    if len(result) != 61:
        print(f"提示：当前附件筛选得到 {len(result)} 个候选单品：原论文数据得到 61 个。")
    return result

def optimize_problem3(candidates: pd.DataFrame, maxiter: int, popsize: int) ->
tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict]:
    n = len(candidates)
    if n < MIN_ITEM_COUNT:
        raise ValueError(f"候选单品数 {n} 小于最少上架数 {MIN_ITEM_COUNT}。")
    baseline = candidates["baseline_item_demand_kg"].to_numpy(dtype=float,
copy=True)
    beta = candidates["item_price_beta"].to_numpy(dtype=float, copy=True)
    reference_price = candidates["reference_price"].to_numpy(dtype=float, copy=True)
    cost = candidates["procurement_cost"].to_numpy(dtype=float, copy=True)
    loss = candidates["loss_rate"].to_numpy(dtype=float, copy=True)
    categories = candidates["category_name"].astype(str).to_numpy(copy=True)
    unique_categories = list(dict.fromkeys(categories))

    price_lower = np.maximum(
        cost * 1.01 / np.maximum(1.0 - loss, 1e-6), 0.80 * reference_price
    )
    price_upper = np.maximum(price_lower * 1.02, 1.20 * reference_price)

    def decode(vector: np.ndarray):
        priorities = vector[:n]
        price_gene = vector[n:2 * n]
        selected_count = int(np.clip(
            np rint(MIN_ITEM_COUNT + (MAX_ITEM_COUNT - MIN_ITEM_COUNT) * vector[-1]),
            MIN_ITEM_COUNT, min(MAX_ITEM_COUNT, n),
        ))
        price = price_lower + price_gene * (price_upper - price_lower)
        demand = np.maximum(0.0, baseline + beta * (price - reference_price))
        replenishment = np.maximum(MIN_DISPLAY_KG, demand / np.maximum(1.0 - loss,
1e-6))
        sales = np.minimum(demand, (1.0 - loss) * replenishment)
        item_profit = price * sales - cost * replenishment

        selected_indices: list[int] = []
        # 每个品类至少保留一个优先级最高的单品。
        for category in unique_categories:

```

```

        indices = np.where(categories == category)[0]
        selected_indices.append(int(indices[np.argmax(priorities[indices])]))
# 再按优先级补足到 27--33 种。
for index in np.argsort(-priorities):
    if len(selected_indices) >= selected_count:
        break
    if int(index) not in selected_indices:
        selected_indices.append(int(index))
selected = np.zeros(n, dtype=bool)
selected[selected_indices] = True
return selected, price, replenishment, demand, sales, item_profit

convergence: list[float] = []

def objective(vector: np.ndarray) -> float:
    selected, _, _, _, profit = decode(vector)
    return -float(profit[selected].sum())

def callback(vector: np.ndarray, convergence_value: float) -> bool:
    del convergence_value
    convergence.append(-objective(vector))
    return False

result = differential_evolution(
    objective,
    bounds=[(0.0, 1.0)] * (2 * n + 1),
    strategy="best1bin",
    maxiter=maxiter,
    popsize=popsize,
    tol=1e-7,
    mutation=(0.5, 1.0),
    recombination=0.85,
    seed=SEED,
    callback=callback,
    polish=True,
    workers=1,
    updating="immediate",
)

if not convergence:
    convergence.append(-float(result.fun))
elif abs(convergence[-1] + float(result.fun)) > 1e-9:
    convergence.append(-float(result.fun))

```

```

selected, price, replenishment, demand, sales, profit = decode(result.x)
all_candidates = candidates.copy()
all_candidates["selected"] = selected.astype(int)
all_candidates["optimal_price_cny_per_kg"] = price
all_candidates["optimal_replenishment_kg"] = np.where(selected, replenishment,
0.0)
all_candidates["expected_demand_kg"] = np.where(selected, demand, 0.0)
all_candidates["expected_sales_kg"] = np.where(selected, sales, 0.0)
all_candidates["expected_profit_cny"] = np.where(selected, profit, 0.0)
all_candidates["price_lower_bound"] = price_lower
all_candidates["price_upper_bound"] = price_upper

selected_items = all_candidates.loc[all_candidates["selected"] ==
1].sort_values(
    "expected_profit_cny", ascending=False
)
convergence_table = pd.DataFrame({
    "iteration": np.arange(1, len(convergence) + 1),
    "best_profit_cny": convergence,
})
summary = {
    "candidate_count": int(n),
    "selected_count": int(selected.sum()),
    "total_replenishment_kg":
float(selected_items["optimal_replenishment_kg"].sum()),
    "expected_sales_kg": float(selected_items["expected_sales_kg"].sum()),
    "max_expected_profit_cny":
float(selected_items["expected_profit_cny"].sum()),
    "de_iterations": int(result.nit),
    "de_function_evaluations": int(result.nfev),
    "de_success": bool(result.success),
    "de_message": str(result.message),
    "constraints_checked": {
        "selected_between_27_and_33": bool(MIN_ITEM_COUNT <= selected.sum() <=
MAX_ITEM_COUNT),
        "minimum_display_2_5kg": bool(
            (selected_items["optimal_replenishment_kg"] >= MIN_DISPLAY_KG - 1e-
9).all()
        ),
        "all_six_categories_represented":
bool(selected_items["category_name"].nunique() == 6),
    },

```

```

    }
    return all_candidates, selected_items, convergence_table, summary

def main() -> None:
    args = parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)

    print("读取附件和问题二结果.....")
    data = load_inputs(args.data_dir, args.q2_results_dir)
    candidates = build_candidate_parameters(data)
    write_csv(candidates, args.output_dir / "q3_candidate_parameters.csv")

    print(f"候选单品数: {len(candidates)}, 开始差分进化优化.....")
    all_candidates, selected_items, convergence, summary = optimize_problem3(
        candidates, maxiter=args.de_maxiter, popsize=args.de_popsize
    )
    write_csv(all_candidates, args.output_dir / "q3_all_candidates.csv")
    # 保留旧文件名, 兼容之前的绘图代码。
    write_csv(all_candidates, args.output_dir / "q3_all_61_candidates.csv")
    write_csv(selected_items, args.output_dir / "q3_2023-07-01_selected_items.csv")
    write_csv(convergence, args.output_dir / "q3_de_convergence.csv")
    (args.output_dir / "q3_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
    )

    export_excel_workbook(
        args.output_dir, candidates, all_candidates, selected_items, convergence,
        summary
    )
    plot_problem3_results(args.output_dir, all_candidates, selected_items,
        convergence)

    print("\n 问题三计算完成: ")
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    print(f"结果目录: {args.output_dir}")

if __name__ == "__main__":
    main()

```